الجمهورية العربية السورية

الجامعة السورية الخاصة

كلية هندسة المعلوماتية

نظام إدارة مركز صحي

اعداد:

نصر ابو قويدر

محمد براء العمار

اشراف :

د.شادي بليدي

حزيران /2026

Syrian Arab Republic

Syrian Private University

Faculty of Information Engineering

“Medical Center Management System”

Prepared by:

Nasr abo Kweder

Mohammad Baraa Al-Ammar

Supervised by:

Dr. Shadi Blidi

June /2026

الملخص

تم تصميم نظام إدارة المركز الطبي لأتمتة وتنظيم العمليات الإدارية والطبية داخل المركز الصحي. يركز المشروع على تطوير نظام يساعد في إدارة المرضى والمواعيد والطواقم الطبية والأنشطة التشغيلية اليومية.

تواجه العديد من المراكز الطبية تحديات تتعلق بالسجلات اليدوية، وبطء سير العمل، والأخطاء البشرية، وفقدان الوقت. يهدف النظام المقترح إلى تحسين الكفاءة من خلال توحيد جميع البيانات في قاعدة بيانات واحدة، مما يتيح وصولاً أسرع للمعلومات، وتحديثاً أسهل، وعمليات متابعة أفضل

كما يدعم النظام تنظيم أدوار المستخدمين المختلفة مثل المسؤولين وموظفي الاستقبال والأطباء و المرضى مع تحديد صلاحيات وصول واضحة لضمان أمن البيانات وخصوصية المرضى.

يساهم هذا النظام في تحسين كفاءة سير العمل، وتحسين جودة الخدمات الصحية، وتقليل الأخطاء الإدارية، وخلق بيئة عمل أكثر تنظيماً ودقةً وفعالية من حيث الوقت.

ABSTRACT

This medical center management system is designed to automate and organize administrative and medical operations within a medical center. The project focuses on developing a system that helps with the management of patients, appointments, medical staff, and daily operational activities. Many medical centers face challenges related to manual record keeping, slow workflows, human errors, and time loss.

The proposed system aims to improve efficiency by centralizing all data in a single database, enabling faster access, easier updates, and better follow-up processes. It also supports the organization of different user roles such as administrators, receptionists, doctors and patients with clearly defined access permissions to ensure data security and patient privacy.

This system contributes to improve the workflow efficiency, improving the quality of healthcare services, reducing administrative errors, and creating a more organized, accurate, and time-efficient working environment

Contents

Chapter One
Introduction

1.1Introduction

With the rapid development of technology and the growing reliance on electronic systems across many sectors, there is a clear need to improve the administrative and medical processes inside medical centers. This project is presented as a solution to the challenges faced by medical centers that still depend on traditional paper-based methods for managing patients, appointments, and medical staff

This introduction aims to provide a brief explanation of the main problem that served as the primary motivation for developing this system, namely the slow completion of transactions, the high rate of human errors, and the difficulty of organizing medical and administrative data using manual methods. It also highlights the shortcomings of current traditional systems that fail to meet the demands of modern operations within medical centers.

The proposed system seeks to achieve the primary goal of improving the efficiency of daily operations and facilitating the management of patients, appointments, and medical records, thereby saving time and effort while enhancing the quality of healthcare services provided. It also aims to support coordination among the various parties working within the medical center, such as administrators, reception staff, and doctors.

This introduction also includes an analysis of the target environment for implementing the system, focusing on the needs of medical centers and the system's beneficiaries, whether they are medical staff or patients. It further emphasizes the importance of the project and the benefits it can offer, such as reducing administrative errors, enhancing data accuracy and integrity, and ensuring the privacy of patient information through an integrated security and authorization system.

Additionally, the introduction clearly defines the scope of the project, indicating the technical and human resources that will be utilized for its implementation, as well as outlining the system's limitations and functionalities. Moreover, it provides a brief review of key previous studies related to medical center management systems, highlighting how they have been utilized in designing and developing the current system.

Finally, the introduction presents the theoretical and scientific foundation supporting the project, explaining the qualitative value this system adds compared to traditional systems and its role in improving administrative and medical performance within medical centers through modern technological solutions

1.2The problem

Medical centers face numerous challenges in managing daily operations, including organizing patient records, managing medical appointments, tracking health conditions, and coordinating work between medical and administrative staff. Many of these centers rely on traditional, paper-based methods for managing information, which leads to slow processes and difficulty in accessing data quickly and accurately.

Additionally, administrative errors resulting from manual data entry can lead to serious issues, such as appointment conflicts, loss or duplication of patient records, and delays in providing medical services, all of which negatively impact the quality of healthcare and patient satisfaction. Other challenges include difficulty in managing time efficiently and the lack of a centralized system for tracking appointments, medical records, and payments.

Medical centers also suffer from weak coordination and communication among different stakeholders, such as reception staff, doctors, and management, which increases the likelihood of errors and places additional pressure on the staff. Therefore, there is a clear need for an integrated electronic system that helps organize daily operations, improve data accuracy, and facilitate internal communication to ensure a more efficient and reliable workflow.

1.3 The proposed solution

This project proposes the design and development of a comprehensive business automation system for managing medical centers, aimed at organizing administrative and medical operations and improving the efficiency of daily work. The proposed system relies on modern technologies that ensure ease of use, data accuracy, and fast access to information. The system includes the following features:

1. **Patient Management**

- A centralized system for registering and managing all patient data, including personal information, medical records, and visit history.
- The ability to quickly search for patients and update their data in an organized and accurate manner.

2. **Appointment Management**

- An integrated electronic calendar for managing examination and consultation appointments.
- Support for booking, modifying, and canceling appointments.
- Sending automatic notifications and reminders to doctors or patients to avoid scheduling conflicts.

3. **Medical Staff Management**

- Managing doctors' accounts and defining their specialties and working hours within the medical center.
- Enabling doctors to view their schedules and add medical notes related to patients.

4. **Medical Records Management**

- An electronic system for storing and archiving medical records in an organized and secure manner.
- Ensuring the protection of medical data through access control mechanisms and privacy preservation.

5. **Reports and Payments**

- Recording payments.
- Generating periodic reports that help the center's management monitor performance and improve service quality.

1.4 Project idea

The proposed system unifies the management of administrative and medical operations within the medical center into a single platform, which contributes to accelerating daily procedures and reducing errors resulting from reliance on traditional methods. The system aims to organize patient data, appointments, medical staff, and medical records in an integrated manner that facilitates access to information and improves work efficiency.

Regarding patient management, the system provides the ability to record all patient-related data, such as personal information, medical history, and previous appointments. It also allows continuous monitoring of the patient's health condition and updating of their data, while ensuring that this information is easily accessible only to authorized parties.

As for appointment management, the system enables precise scheduling of medical examinations and consultations, with the ability to set important appointments and organize doctors' daily schedules. It also sends automatic notifications and alerts to doctors and reception staff to avoid scheduling conflicts and ensure adherence to scheduled appointments.

In terms of medical records, the system allows secure and organized storage within a centralized database, making them easily accessible according to the patient. It also provides effective search capabilities within medical records, which helps save time and improve the speed of delivering medical services.

Moreover, the system allows the generation of periodic reports related to the number of patients, appointments, payments, and the overall performance of the medical center, helping management monitor performance and make data-driven decisions. Data protection is ensured through the implementation of modern security techniques, such as data encryption and role-based access control, ensuring the privacy of medical information.

1.5 Project goal

The objectives of the medical center management system project are to improve the operational efficiency of the center by organizing all daily administrative and medical activities within a unified electronic platform. The system aims to facilitate the management of patient affairs, appointments, and medical records, which helps reduce human errors and accelerate the execution of various processes within the medical center.

The system also seeks to enhance coordination and communication between medical and administrative staff by providing structured tools for managing appointments, tracking medical cases, and exchanging information in a clear and effective manner. It further aims to simplify the tracking of payments and administrative reports, helping improve resource management and support informed decision-making.

The project also focuses on strengthening security and protecting sensitive medical data by implementing advanced mechanisms for access control and data encryption, ensuring patient privacy and compliance with established healthcare standards and regulations. By achieving these objectives, the system contributes to increasing productivity, reducing operational costs, and improving patient experience and the quality of healthcare services provided.

1.6 Project methodology

The Iterative Incremental Development Methodology was adopted in implementing the medical center management system project. This is one of the commonly used methodologies in software development and project management. It is based on dividing the system into a set of smaller parts or versions (incremental iterations), where each part is developed gradually with added improvements and enhancements in each new iteration.

This methodology is built on the principle of progressive system development, where in each iteration a set of core functionalities is developed, then tested and evaluated, and subsequently improved based on feedback before moving on to the next iteration. This approach helps in identifying errors at early stages, improving system quality, and ensuring alignment with user requirements.

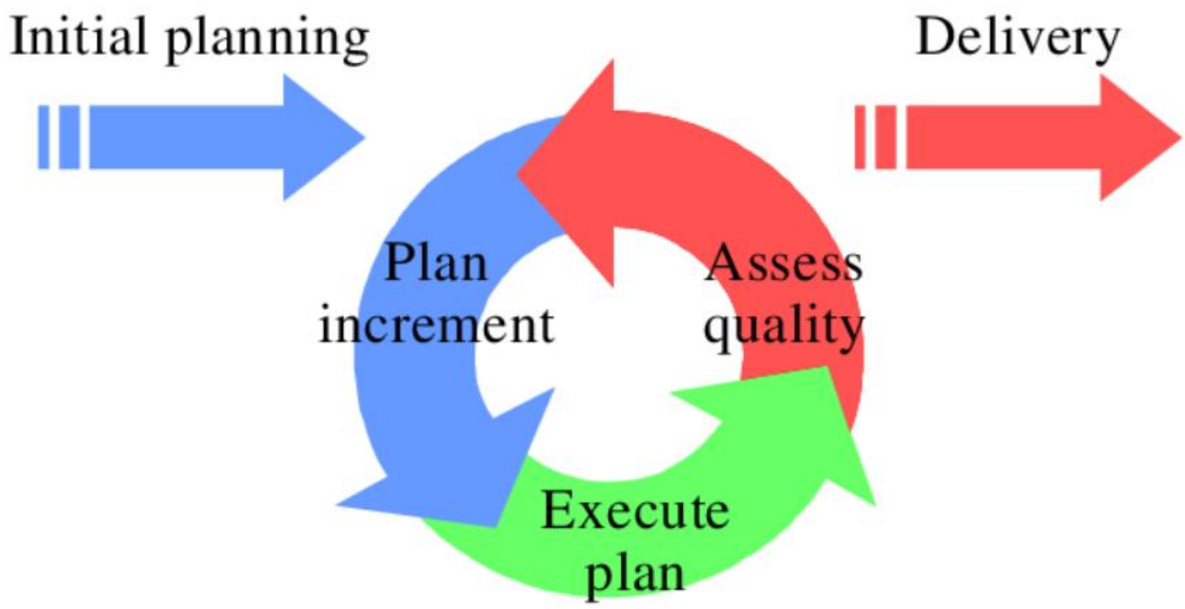


Figure (1.6.1): Incremental and iterative development model

Figure (1.6.1) illustrates the steps of the incremental iterative development methodology adopted in this project, where the work was divided into several successive iterations, with each iteration beginning by adding new functionalities or improving previous functionalities, leading to the construction of a complete and ready-to-use system.

Explanation of the chart elements:

• **Iterations (Iteration 1, Iteration 2, Iteration 3):** These represent sequential stages in the system development process, where each iteration adds new functionalities such as user management, patient management, appointment management, and medical records.

• **Horizontal arrows:** These indicate the gradual transition between iterations, reflecting the continuous development process of the system.

• **Feedback:** This represents the review and evaluation processes conducted after each iteration, aimed at improving performance and correcting errors before moving to the next stage.

• **Final Product:** This represents the integrated system achieved after completing all iterations, which becomes ready for use within the medical center.

This methodology helps ensure development flexibility, improve system quality, and better meet user requirements, while reducing the risks associated with developing complex software systems.

1.7 Summary

This chapter provides a brief description of the medical center management system, where the project objectives and its importance in improving the efficiency of daily operations and facilitating the management of patients, appointments, and medical records have been identified. It also clarifies the project boundaries and scope, along with the challenges and problems that were identified and formed the main motivation for developing and proposing this system.

The chapter also discusses the adopted development methodology, which is the Iterative & Incremental Development methodology. This approach aims to ensure efficient and effective project implementation, with the possibility of continuous evaluation and improvement throughout each development stage. It contributes to ensuring system quality, reducing errors, and achieving the desired objectives in a gradual and organized manner.

Through this chapter, the reader becomes familiar with the project idea, the problems it addresses, its objectives, and the importance of its implementation in medical center environments, which lays the foundation for understanding the remaining system details and functionalities in the following chapters.

Chapter Two

Research and Study Topic Reference

2.1 Introduction

This chapter addresses the clarification of the main fundamental concepts related to the medical center management system, with a focus on how to automate and organize daily administrative and medical processes in a more efficient and professional manner. The chapter aims to introduce the reader to the terminology and technologies used in healthcare management systems, as well as to explain the components of the proposed system and its objectives.

It also highlights how integration between the electronic system and the operational needs of medical centers is achieved, ensuring improved quality of healthcare services, facilitating the management of patients and appointments, and reducing administrative and medical errors resulting from reliance on traditional methods.

2.2 Basic Concepts

Patient Management
This is the process of registering and organizing patient data within the system, including personal information, medical history, previous visits, and health conditions. This concept aims to facilitate access to patient data and improve the accuracy of medical documentation.

Medical Appointment Management
This involves scheduling and organizing examination and consultation appointments, while defining doctors' working hours and avoiding scheduling conflicts. The system also supports sending automatic notifications to remind patients and doctors of scheduled appointments.

Medical Staff Management
This concept includes organizing the data of doctors and medical staff, defining their specialties, working hours, and system permissions. This helps improve coordination between medical and administrative staff.

Medical Records Management
The system provides the ability to store and archive medical records electronically, enabling easy, secure, and organized access to patient information. This concept supports the protection of medical data and ensures quick access when needed.

Medical and Administrative Reports
The system enables the generation of periodic reports related to the number of patients, appointments, medical performance, and payments. This helps the medical center's management monitor operations and make decisions based on accurate data.

.

2.3 Reference study

Practo Platform
It is a global electronic platform aimed at facilitating the management of clinics and medical centers. It provides services for online appointment booking, patient data management, and doctor schedule organization. The platform helps patients easily access medical services, while also supporting doctors in efficiently managing their clinics.

Clinic Management System
This is an electronic system designed specifically for managing medical clinics. It enables patient registration, appointment management, medical record storage, and invoice generation. The system features a user-friendly interface, which helps improve workflow and reduce reliance on paper records.

Zocdoc Platform
This platform provides online medical appointment booking services, allowing patients to search for doctors based on specialty and location and book appointments easily. It also helps doctors organize their schedules and improve the efficiency of communication with patients.

2.3.1 Comparison between the previous platforms and the current platform

The previous systems are similar in providing appointment and patient management services, but the system proposed in this project is distinguished by offering an integrated platform that combines medical and administrative management in a single system. The current system also focuses on simplifying the internal operations of the medical center, enhancing the security of medical data, and customizing permissions according to user roles, making it more suitable for the needs of local medical centers compared to general platforms.

Feature	Angel Infotech CMS	Heltog CMS	Ray by Practo	Our system
Add new patient	✓	✓	✓	✓
Edit patient information	✓	✗	✗	✓
Search patient	✗	✓	✗	✓
Book appointment	✓	✓	✓	✓
Edit appointment	✓	✓	✗	✓
Cancel appointment	✓	✗	✓	✓
View doctor's daily schedule	✓	✗	✓	✓
Manage doctor availability	✗	✓	✓	✓
Add medical notes for patient	✗	✓	✗	✓
View monthly reports	✓	✗	✓	✓

2.4 Components of the proposed system

1. Periodic Evaluation

The system provides a mechanism for periodic evaluation of the medical center's performance by monitoring daily workflow, appointment management efficiency, and the quality level of healthcare services provided. It also enables the preparation of regular evaluation reports that help management identify strengths and weaknesses and continuously improve medical and administrative processes.

2. Analytical Reports
The system supports the generation of monthly or quarterly analytical reports that illustrate the performance of the medical center in terms of the number of patients and completed appointments. These reports provide accurate statistics that help doctors and management make decisions based on reliable data.

3. Management Reports
The system provides comprehensive administrative reports related to productivity, human and medical resource management, and overall performance monitoring of the medical center. These reports also include indicators that help management assess the level of work, provide recommendations for improving performance, and ensure continuous development.

Chapter Three

Analytical Study of the Proposed System

3.1 Introduction

In this chapter, a comprehensive analytical study of the medical center management system will be presented, where the technical and financial feasibility of the project is evaluated from several perspectives. This study includes an analysis of the available technical resources and an assessment of the possibility of implementing the proposed system within the medical center environment, taking into consideration the nature of medical and administrative work.

It will also evaluate the level of acceptance of the system by medical staff, administrative staff, and patients, as well as its impact on improving the efficiency of daily operations and the quality of healthcare services provided. In addition, a Gantt Chart will be presented to illustrate the project timeline and the distribution of key tasks over specific time periods.

A Requirements Document will also be provided, defining the functional and non-functional requirements of the system, such as patient management, appointment management, medical records, and user management. Furthermore, a set of detailed diagrams will be presented to illustrate the system structure, such as the Use Case Diagram, which shows user interaction with the system, and Sequence Diagrams, which illustrate the flow of operations between system components.

Finally, a list of tests for each use case will be presented, with the results summarized in a Requirement Traceability Matrix to ensure that the system meets all specified requirements efficiently and accurately.

3.2 Feasibility study

In this phase, the project feasibility is analyzed by evaluating a set of key factors to determine the possibility of successfully implementing the proposed system within medical centers. These factors include:

1. Economic Feasibility
The total cost of the project is evaluated, including development, maintenance, and training costs, compared to the expected returns from improving work efficiency and reducing operational costs. The economic benefits are also analyzed, such as saving time and effort in managing patients and appointments, and reducing administrative errors that may lead to financial losses.

2. Technical Feasibility
This involves evaluating the available technical capabilities, such as technological infrastructure, databases, and software systems used. It also includes analyzing the development team’s ability to implement the system and its compatibility with existing technologies and systems used within medical centers.

3. Social Feasibility
This evaluates the level of acceptance of the new system by medical staff, administrative staff, and patients, as well as its impact on improving the patient experience and increasing satisfaction. It also examines the role of the system in facilitating communication between patients, reception staff, and doctors.

4. Operational Feasibility
This assesses the system's ability to improve daily operations within the medical center, such as appointment scheduling, medical record management, and coordination between medical and administrative staff. It also analyzes the system's impact on increasing employee productivity and reducing human errors.

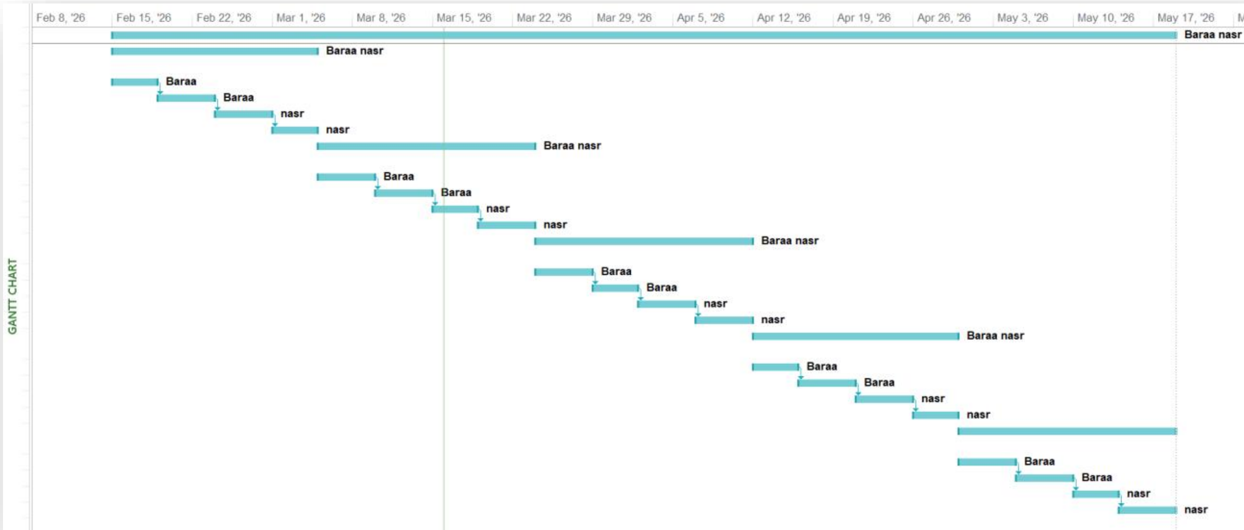
5. Expected Outputs
After completing the feasibility study, a detailed report will be prepared outlining the findings and recommendations. A work plan will also be presented, including the project timeline, required tasks, and proposed budget. The ultimate goal is to ensure that the proposed system is feasible and capable of achieving the desired objectives in terms of improving work efficiency, healthcare service quality, and patient satisfaction.

3.3 Project plan:

3.3.1 GANTT CHART

GANTT CHART

		Task Mode	اسم المهمة	Duration	Start	Finish	Predecessors	Resource Names
1			تجهيز التقرير	80 days	Sun 2/15/26	Mon 5/18/26		Baraa nasr
2			Iteration 1: إدارة المواعيد وتوفر الأطباء	16 days	Sun 2/15/26	Wed 3/4/26		Baraa nasr
3			التحليل	4 days	Sun 2/15/26	Wed 2/18/26		Baraa
4			التصميم	4 days	Thu 2/19/26	Mon 2/23/26	3	Baraa
5			التنفيذ	4 days	Tue 2/24/26	Sat 2/28/26	4	nasr
6			الاختبار	4 days	Sun 3/1/26	Wed 3/4/26	5	nasr
7			Iteration 2: إدارة حسابات المرضى	16 days	Thu 3/5/26	Mon 3/23/26		Baraa nasr
8			التحليل	4 days	Thu 3/5/26	Mon 3/9/26		Baraa
9			التصميم	4 days	Tue 3/10/26	Sat 3/14/26	8	Baraa
10			التنفيذ	4 days	Sun 3/15/26	Wed 3/18/26	9	nasr
11			الاختبار	4 days	Thu 3/19/26	Mon 3/23/26	10	nasr
12			Iteration 3: نظام المحادثة بين المريض والطبيب	16 days	Tue 3/24/26	Sat 4/11/26		Baraa nasr
13			التحليل	4 days	Tue 3/24/26	Sat 3/28/26		Baraa
14			التصميم	4 days	Sun 3/29/26	Wed 4/1/26	13	Baraa
15			التنفيذ	4 days	Thu 4/2/26	Mon 4/6/26	14	nasr
16			الاختبار	4 days	Tue 4/7/26	Sat 4/11/26	15	nasr
17			Iteration 4: ميزات الإداره والتقارير	16 days	Sun 4/12/26	Wed 4/29/26		Baraa nasr
18			التحليل	4 days	Sun 4/12/26	Wed 4/15/26		Baraa
19			التصميم	4 days	Thu 4/16/26	Mon 4/20/26	18	Baraa
20			التنفيذ	4 days	Tue 4/21/26	Sat 4/25/26	19	nasr
21			الاختبار	4 days	Sun 4/26/26	Wed 4/29/26	20	nasr
22			Iteration 5: المراجعات النهائية وإصلاح الأخطاء	16 days	Thu 4/30/26	Mon 5/18/26		
23			التحليل	4 days	Thu 4/30/26	Mon 5/4/26		Baraa
24			التصميم	4 days	Tue 5/5/26	Sat 5/9/26	23	Baraa
25			التنفيذ	4 days	Sun 5/10/26	Wed 5/13/26	24	nasr
26			الاختبار	4 days	Thu 5/14/26	Mon 5/18/26	25	nasr



3.4.1 Problem Definition

Medical centers face many challenges in managing daily operations, which directly affects work efficiency and the quality of healthcare services provided to patients. These challenges include:

1. Data and Medical Records Management
Many medical centers rely on manual methods or non-integrated systems to manage patient records, which leads to time loss, difficulty in accessing information, and an increased likelihood of losing or duplicating important medical data.

2. Medical Appointment Scheduling
Reception staff and doctors face difficulties in effectively organizing medical examination and consultation appointments, which leads to scheduling conflicts, long waiting times, and failure to adhere to planned schedules.

3. Communication with Patients
The medical center suffers from weak effective communication with patients, whether regarding appointment confirmation, sending reminders, or informing patients about updates related to their health condition, which negatively affects patient experience and satisfaction.

4. Follow-up of Medical Cases
The lack of a centralized system for tracking medical cases and continuously updating records makes it difficult for doctors to accurately follow a patient’s medical history, which may lead to delays in providing healthcare or making inaccurate medical decisions.

5. Reports and Analytics
Traditional systems do not provide accurate analytical reports that help medical center management monitor performance, analyze the number of patients, appointments, and revenues, and make data-driven administrative decisions based on reliable information.

3.4.2 The purpose of the system

The purpose of this document is to provide a detailed requirements specification for the medical center management system. The system aims to improve the efficiency of administrative and medical operations by providing integrated tools for managing patients, appointments, medical records, and medical staff. This document also defines the core functionalities of the system, the constraints that must be considered, and the way users interact with the system.

3.4.3 The main functions of the system

1. Patient Management

- Recording patient data and storing it in a centralized and secure manner.
- Monitoring medical history and patient conditions in an organized way.

2. Appointment Management

- Organizing medical examination and consultation appointments.
- Sending automatic reminders and notifications to patients and doctors.

3. Medical Staff Management

- Recording doctors’ data and defining their specialties and working hours.
- Allowing doctors to view their daily schedules and add medical notes.

4. Medical Records Management

- Storing and archiving medical records electronically in a secure manner.
- Enabling quick search of medical records and linking them to patients.

5. Reports and Analytics

- Generating detailed reports on the number of patients, appointments, and overall performance of the medical center.

3.4.4 Target users

1. Doctors

- Using the system to follow up on patients and appointments.
- Accessing medical records and adding medical notes and reports.

2. Patients

- Viewing their medical appointments and receiving notifications and reminders.
- Accessing certain authorized health information through the system.

3. Reception and Administrative Staff

- Managing patient data and daily appointments.
- Managing users and permissions and generating administrative reports.

3.4.5 Functional Requirements

ID	Requirements	Actor
FR-1	Login	Admin, Receptionist, Doctor
FR-2	Logout	Admin, Receptionist, Doctor
FR-3	Add doctor	Admin
FR-4	Delete doctor	Admin
FR-5	Add receptionist	Admin
FR-6	Delete receptionist	Admin
FR-7	View complaints	Admin
FR-8	Add specialization to doctor	Admin
FR-9	Add patient	Receptionist
FR-10	Search for patient	Receptionist
FR-11	Book appointment	Receptionist
FR-12	Edit appointment	Receptionist
FR-13	Cancel appointment	Receptionist
FR-14	Record payments	Receptionist
FR-15	Edit patient information	Receptionist
FR-16	Monthly reports	Receptionist
FR-17	Send notification to doctor	Receptionist
FR-18	View doctor's availability	Receptionist
FR-19	Create account by patient	Patient
FR-20	Edit patient account	Patient

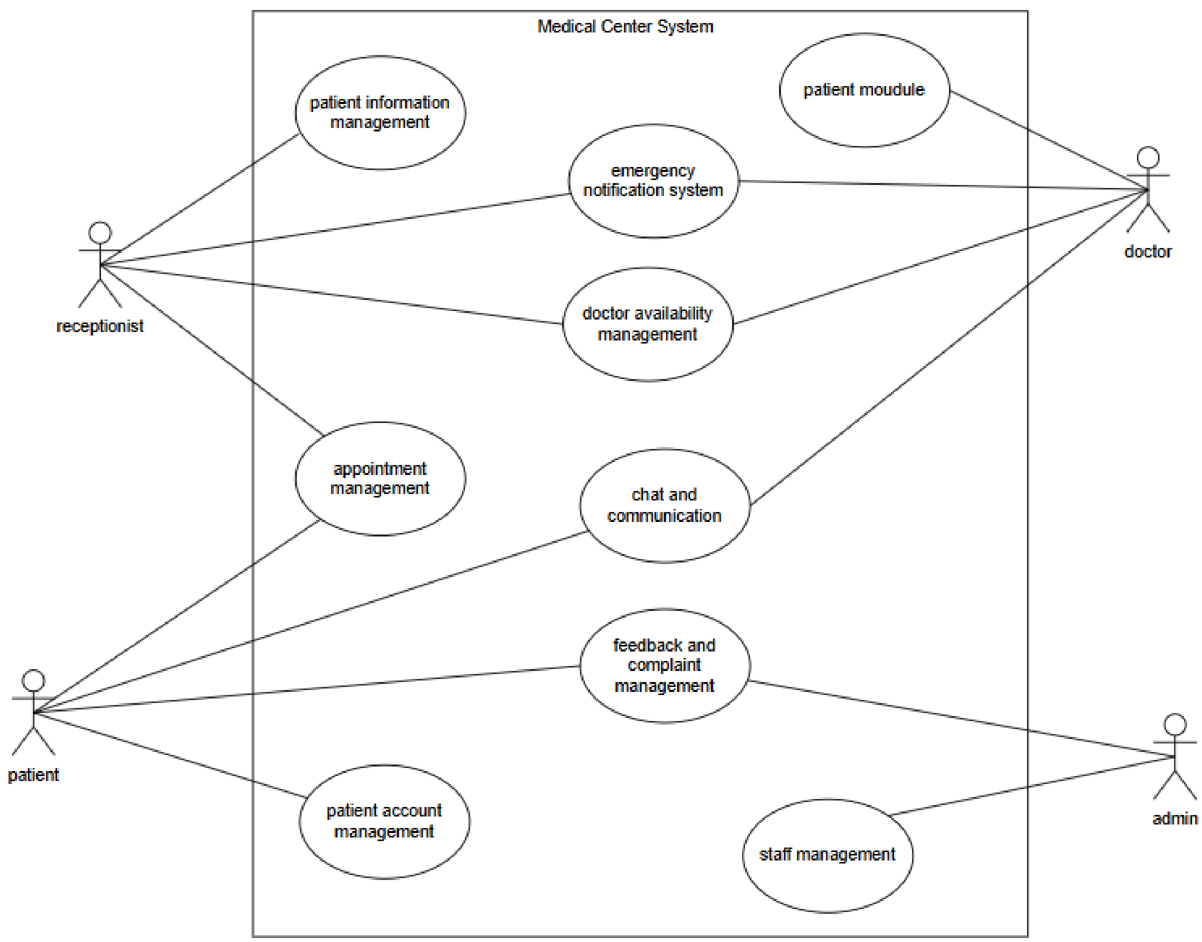
FR-21	Patient views doctors' appointments	Patient
FR-22	Book appointment	Patient
FR-23	Edit appointment	Patient
FR-24	Cancel appointment	Patient
FR-25	Patient receives appointment notifications	Patient
FR-26	Conduct conversation with doctor	Patient
FR-27	Send text messages	Patient
FR-28	Send photos or files	Patient
FR-29	Give rating or complaint	Patient
FR-30	Receive sent messages	Doctor
FR-31	View messages	Doctor
FR-32	Reply to messages	Doctor
FR-33	Set availability times	Doctor
FR-34	View appointments	Doctor
FR-35	View patient information	Doctor
FR-36	Add medical notes	Doctor

3.4.6 Non-Functional Requirements

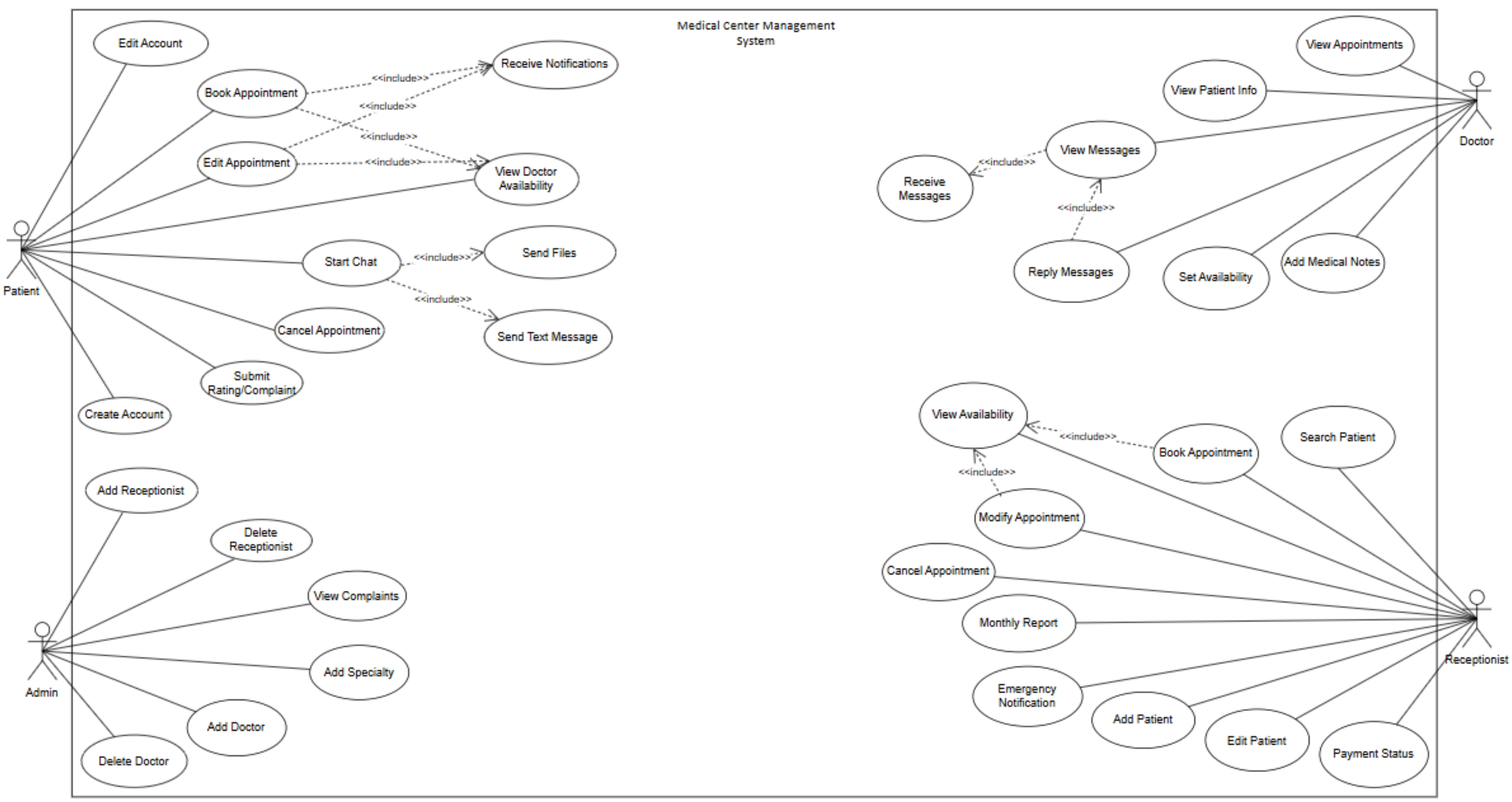
Non-functional requirements define the general qualities the system must have to ensure ease of use, data security, and reliable performance

1. Usability
   The system should be easy to understand and use. Pages, buttons, and messages should be clear so that staff can learn the system quickly.
2. Performance
   Basic actions such as logging in, searching for a patient, or opening the appointments page should respond within a few seconds under normal use.
3. Reliability
   The system should run in a stable way during the medical center's working hours.
4. Security
   - Only logged-in users can access the system.
   - Each role can only see and do what is allowed for that role.
   - Passwords are stored securely in the database.
5. Data Integrity
   All patient, appointment, and payment data should be stored in a single database.

Use case diagram (high level):



Use case diagram (low level):



3.4.7 Use case specification:

Field	Details
Use Case	Login
Use Case ID	UC-01
Actor	Admin, Receptionist, Doctor
Goal	Log into the system using email and password.
Preconditions	User account exists.
Main Flow	1) Open login page 2) Enter email and password 3) System validates 4) System redirects to dashboard based on role
Alternative Flow	If credentials are wrong show error
Postconditions	User is logged in.

Field	Details
Use Case	Logout
Use Case ID	UC-02
Actor	Admin, Receptionist, Doctor
Goal	Log out from the system.
Preconditions	User is logged in.
Main Flow	1) Click Logout 2) System ends session 3) Return to login page
Alternative Flow	None
Postconditions	User is logged out.

Field	Details
Use Case	Add Doctor
Use Case ID	UC-03
Actor	Admin
Goal	Add a new doctor account.
Preconditions	Admin is logged in.
Main Flow	1) Open staff management 2) Click Add Doctor 3)Enter details 4) Save 5) System creates doctor user
Alternative Flow	Missing or duplicate email shows validation error.
Postconditions	Doctor account is created.

Field	Details
Use Case	Delete Doctor
Use Case ID	UC-04
Actor	Admin
Goal	Remove a doctor account.
Preconditions	Admin is logged in and doctor exists.
Main Flow	1) Open doctors list 2) Select doctor 3) Click Delete 4) Confirm 5) System deletes doctor
Alternative Flow	Doctor has appointments system warns.
Postconditions	Doctor removed

Field	Details
Use Case	Add Receptionist
Use Case ID	UC-05
Actor	Admin
Goal	Add a new receptionist account.
Preconditions	Admin is logged in.
Main Flow	1) Open staff management 2) Click Add Receptionist 3) Enter details 4) Save 5) System creates receptionist user
Alternative Flow	None
Postconditions	Receptionist account created.

Field	Details
Use Case	Delete Receptionist
Use Case ID	UC-06
Actor	Admin
Goal	Remove a receptionist account.
Preconditions	Admin is logged in and receptionist exists.
Main Flow	1) Open receptionists list 2) Select receptionist 3) Click Delete 4) Confirm 5) System deletes receptionist
Alternative Flow	None
Postconditions	Receptionist removed.

Field	Details
Use Case	Add Patient
Use Case ID	UC-07
Actor	Receptionist
Goal	Register a new patient.
Preconditions	Receptionist is logged in.
Main Flow	1) Open Patients 2) Click Add Patient 3) Fill details 4) Save 5) System stores patient record
Alternative Flow	Missing data shows validation error.
Postconditions	Patient is created.

Field	Details
Use Case	Edit Patient
Use Case ID	UC-08
Actor	Receptionist
Goal	Update patient information.
Preconditions	Patient exists.
Main Flow	1) Search/select patient 2) Edit details 3) Save 4) System updates record
Alternative Flow	None
Postconditions	Patient updated.

Field	Details
Use Case	Search Patient
Use Case ID	UC-09
Actor	Receptionist
Goal	Find a patient record quickly.
Preconditions	Receptionist is logged in
Main Flow	1) Enter search keyword 2) System shows matches 3) Select patient from results
Alternative Flow	No results then show "No results found".
Postconditions	Patient list displayed.

Field	Details
Use Case	Book Appointment
Use Case ID	UC-10
Actor	Receptionist
Goal	Create an appointment for a patient with a doctor.
Preconditions	Patient exists and doctor exists.
Main Flow	1) Open Appointments Create 2) Select patient 3) Select doctor 4) Choose date and time 5) System checks availability and conflicts 6) Save appointment
Alternative Flow	If Slot unavailable then show warning then choose another slot.
Postconditions	Appointment created

Field	Details
Use Case	Modify Appointment
Use Case ID	UC-11
Actor	Receptionist
Goal	reschedule an appointment.
Preconditions	Appointment exists.
Main Flow	1) Open appointment 2) Change time slot 3) System checks conflicts 4) Save changes
Alternative Flow	New slot unavailable then show message and don't save.
Postconditions	Appointment updated.

Field	Details
Use Case	Cancel Appointment
Use Case ID	UC-12
Actor	Receptionist
Goal	Cancel an appointment.
Preconditions	Appointment exists.
Main Flow	1) Select appointment 2) Click Cancel 3) Confirm 4) System sets status to cancelled
Alternative Flow	None
Postconditions	Appointment cancelled.

Field	Details
Use Case	View Doctor Availability
Use Case ID	UC-13
Actor	Receptionist
Goal	View available slots for a doctor.
Preconditions	Doctor availability exists
Main Flow	1) Select doctor 2) System shows available slots
Alternative Flow	No slots → show “No available slots”.
Postconditions	Availability shown.

Field	Details
Use Case	Mark Paid/Unpaid
Use Case ID	UC-14
Actor	Receptionist
Goal	Mark an appointment as paid or unpaid using checkbox.
Preconditions	Appointment exists.
Main Flow	1) Open appointment edit 2) Check/uncheck Paid 3) Save 4) System updates
Alternative Flow	None
Postconditions	Payment status saved on appointment.

Field	Details
Use Case	Generate Monthly Report
Use Case ID	UC-15
Actor	Receptionist
Goal	View monthly appointment summary + paid/unpaid counts.
Preconditions	Receptionist is logged in
Main Flow	1) press generate reports 2) Select month 3) System makes report
Alternative Flow	No data then show empty report message.
Postconditions	Report displayed.

Field	Details
Use Case	Emergency Notification to Doctor
Use Case ID	UC-16
Actor	Receptionist
Goal	Send an emergency alert to a doctor.
Preconditions	Doctor exists.
Main Flow	1) Select doctor 2) Write message 3) Send 4) System delivers notification
Alternative Flow	None
Postconditions	Notification sent.

Field	Details
Use Case	View Appointments
Use Case ID	UC-17
Actor	Doctor
Goal	View assigned appointments.
Preconditions	Doctor is logged in.
Main Flow	1) Open My Appointments 2) System lists upcoming appointments
Alternative Flow	No appointments → show empty list message.
Postconditions	Appointments displayed.

Field	Details
Use Case	View Patient Information
Use Case ID	UC-18
Actor	Doctor
Goal	View patient details for treatment.
Preconditions	Patient exists and linked to appointment.
Main Flow	1) Open appointment/patient 2) System shows patient info
Alternative Flow	None
Postconditions	Patient info displayed.

Field	Details
Use Case	Set Availability
Use Case ID	UC-19
Actor	Doctor
Goal	Set working days and hours.
Preconditions	Doctor is logged in.
Main Flow	1) Open Availability 2) Select days and time range 3) Save 4) System stores availability
Alternative Flow	None
Postconditions	Availability saved.

Field	Details
Use Case	Add Medical Notes
Use Case ID	UC-20
Actor	Doctor
Goal	Add notes for a patient
Preconditions	Doctor logged in and patient exists.
Main Flow	1) Open patient/appointment 2) Write note 3) Save 4) System stores medical note
Alternative Flow	Empty then show validation error.
Postconditions	Medical note saved.

Field	Details
Use Case	Create patient account
Use Case ID	FR-21
Description	The patient creates a new account on the system
Actor	Patient
Precondition	The patient does not have an existing account
Postcondition	The patient account is created and stored in the system
Main Scenario	1. Open the registration page 2. Enter personal information (name, email, password, phone) 3. Press the register button
Alternative Scenario	Invalid or incomplete data, the user returns to step 2

Field	Details
Use Case	Edit patient account
Use Case ID	FR-22
Description	The patient edits their account information
Actor	Patient
Precondition	The patient is logged in
Postcondition	The account data is updated
Main Scenario	1. Open profile page 2. Edit the desired information 3. Save
Alternative Scenario	Invalid data, the user returns to step 2

Field	Details
Use Case	View doctor availability
Use Case ID	FR-23
Description	The patient views available times for doctors
Actor	Patient
Precondition	1. The patient is logged in 2. Doctors have set their availability
Postcondition	Display available times
Main Scenario	1. Open the doctors page 2. Select a doctor 3. View available times
Alternative Scenario	No available times

Field	Details
Use Case	Book appointment
Use Case ID	FR-24
Description	The patient books an appointment with a doctor
Actor	Patient
Precondition	1. The patient is logged in 2. Doctor has available times
Postcondition	The appointment is registered
Main Scenario	1. Select a doctor 2. Select an available time 3. Confirm the booking
Alternative Scenario	Time conflict, the user returns to step 2

Field	Details
Use Case	Edit appointment
Use Case ID	FR-25
Description	The patient edits an existing appointment
Actor	Patient
Precondition	An appointment exists for the patient
Postcondition	The appointment is updated
Main Scenario	1. Open appointments page 2. Select the appointment 3. Edit the details 4. Save
Alternative Scenario	Time conflict, the user returns to step 3

Field	Details
Use Case	Cancel appointment
Use Case ID	FR-26
Description	The patient cancels an existing appointment
Actor	Patient
Precondition	An appointment exists for the patient
Postcondition	The appointment is deleted
Main Scenario	1. Open appointments page 2. Select the appointment 3. Press cancel 4. Confirm cancellation
Alternative Scenario	None

Field	Details
Use Case	Receive appointment notifications
Use Case ID	FR-27
Description	The patient receives notifications about their appointments
Actor	Patient
Precondition	An appointment exists for the patient
Postcondition	The notification is displayed to the patient
Main Scenario	1. The system automatically sends a notification to the patient 2. The notification displays appointment details
Alternative Scenario	Notification delivery failure

Field	Details
Use Case	Start chat with doctor
Use Case ID	FR-28
Description	The patient starts a chat conversation with a doctor
Actor	Patient
Precondition	1. The patient is logged in 2. The patient has booked an appointment with the doctor
Postcondition	Chat window is opened
Main Scenario	1. Select the doctor 2. Press the chat button 3. Chat window opens
Alternative Scenario	No appointment with the doctor

Field	Details
Use Case	Send text messages
Use Case ID	FR-29
Description	The patient sends a text message to the doctor
Actor	Patient
Precondition	1. An open chat with the doctor exists 2. The patient has a current or past appointment with the doctor
Postcondition	The message is sent and saved
Main Scenario	1. Type the message 2. Press send
Alternative Scenario	Send failure, the user retries

Field	Details
Use Case	Send images or files
Use Case ID	FR-30
Description	The patient sends images or files to the doctor
Actor	Patient
Precondition	1. An open chat with the doctor exists 2. The patient has a current or past appointment with the doctor
Postcondition	The file is sent and saved
Main Scenario	1. Press the attach button 2. Select the image or file 3. Press send
Alternative Scenario	Unsupported file type, the user returns to step 2

Field	Details
Use Case	Submit rating or complaint
Use Case ID	FR-31
Description	The patient submits a rating or files a complaint
Actor	Patient
Precondition	The patient has a previous appointment
Postcondition	The rating or complaint is saved
Main Scenario	1. Open the rating page 2. Select a rating or write a complaint 3. Press submit
Alternative Scenario	Incomplete data, the user returns to step 2

Field	Details
Use Case	Receive sent messages
Use Case ID	FR-32
Description	The doctor receives messages sent by the patient
Actor	Doctor
Precondition	1. Logged in 2. A message has been sent by a patient with a current or past appointment
Postcondition	A new message notification is displayed
Main Scenario	1. The system sends a notification to the doctor 2. The notification displays the patient's name
Alternative Scenario	None

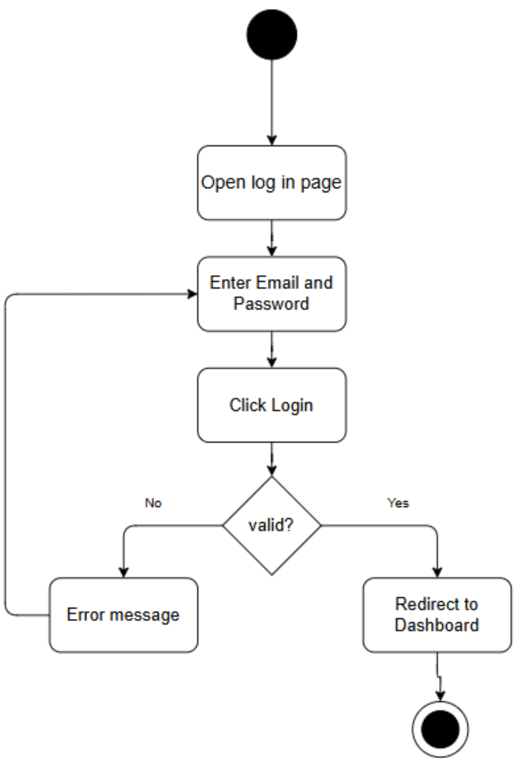
Field	Details
Use Case	View messages
Use Case ID	FR-33
Description	The doctor views messages sent by patients
Actor	Doctor
Precondition	1. Logged in 2. Messages exist from patients with current or past appointments
Postcondition	Messages are displayed
Main Scenario	1. Open the messages page 2. Select the conversation 3. View messages
Alternative Scenario	No messages

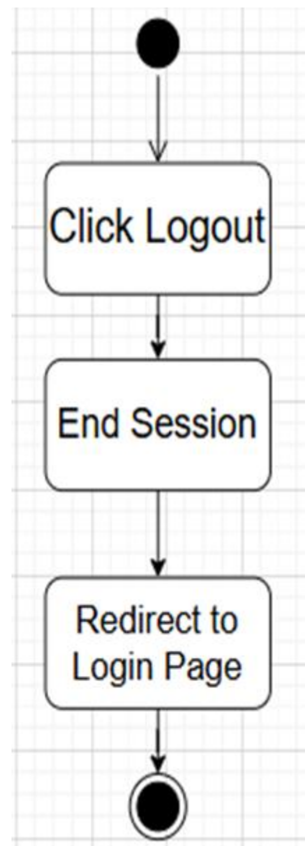
Field	Details
Use Case	Reply to messages
Use Case ID	FR-34
Description	The doctor replies to a patient's message
Actor	Doctor
Precondition	A message from a patient with a current or past appointment exists
Postcondition	The reply is sent and saved
Main Scenario	1. Open the conversation 2. Type the reply 3. Press send
Alternative Scenario	Send failure, the user retries

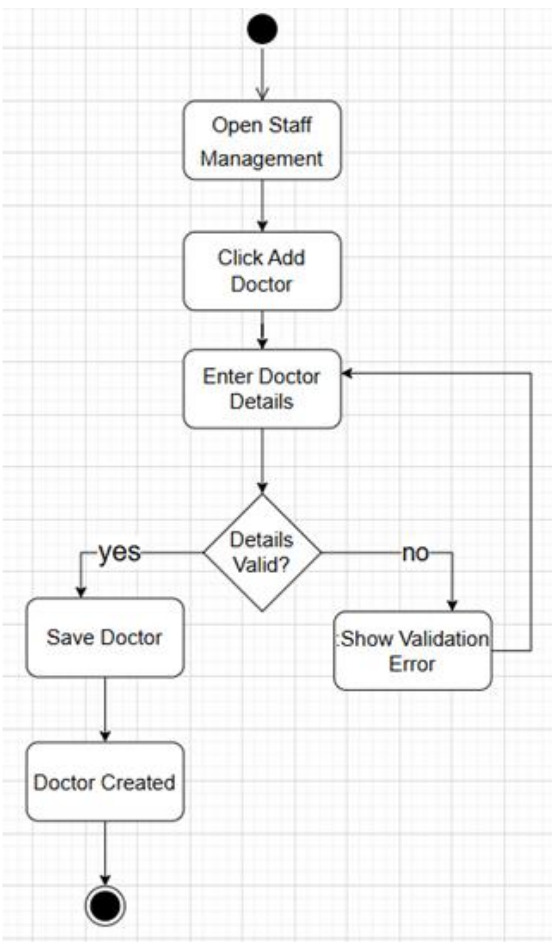
Field	Details
Use Case	View complaints
Use Case ID	FR-35
Description	The admin views complaints submitted by patients
Actor	Admin
Precondition	Complaints exist in the system
Postcondition	Complaints are displayed
Main Scenario	1. Open the complaints page 2. View complaint details
Alternative Scenario	No complaints

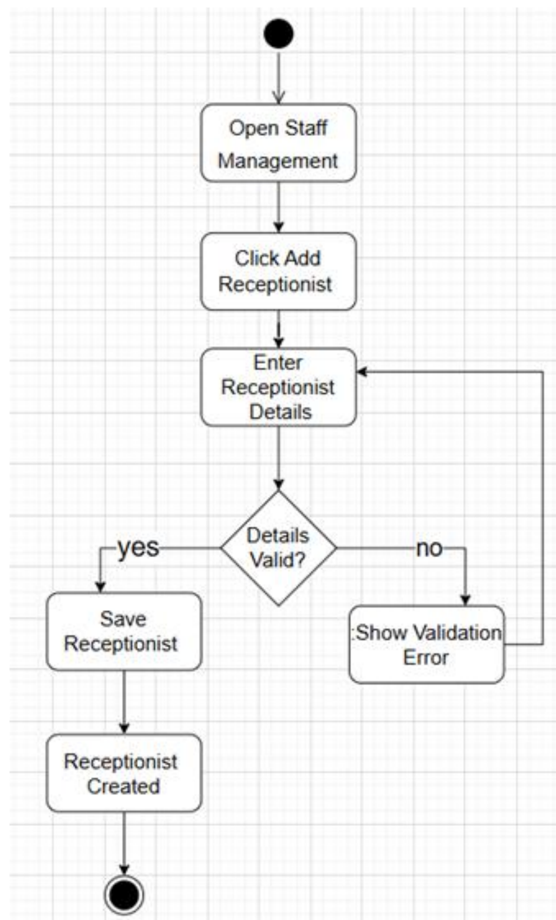
Field	Details
Use Case	Add doctor specialty
Use Case ID	FR-36
Description	The admin assigns a specialty to a doctor
Actor	Admin
Precondition	The doctor exists in the system
Postcondition	The specialty is saved
Main Scenario	1. Select the doctor 2. Add or select the specialty 3. Save
Alternative Scenario	Invalid data, the user returns to step 2

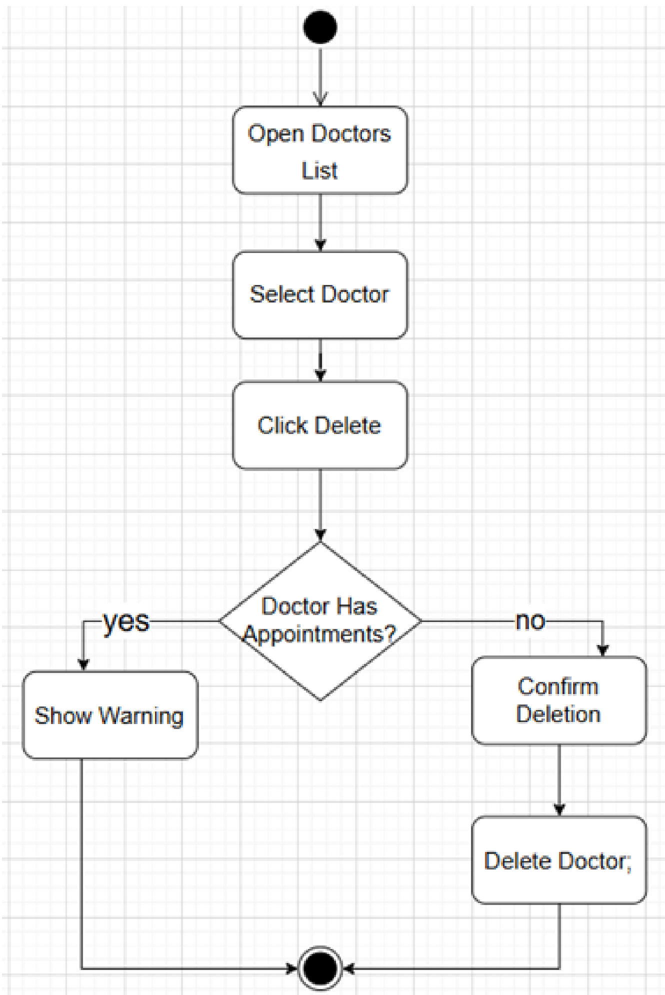
3.4.8 Activity Diagram :

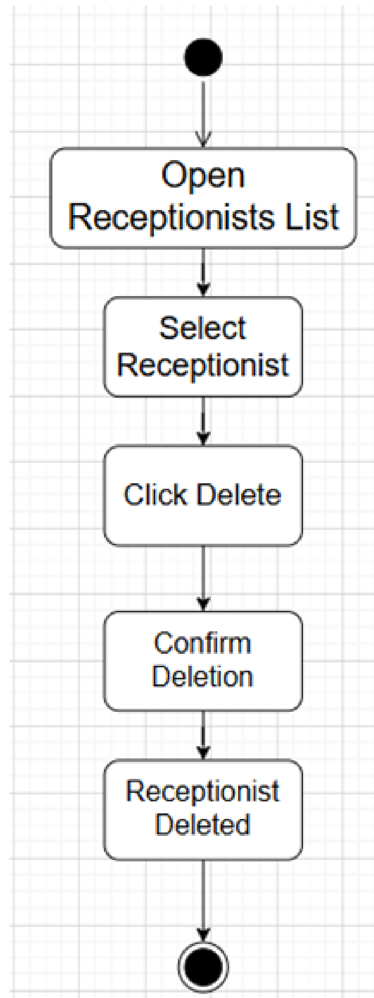


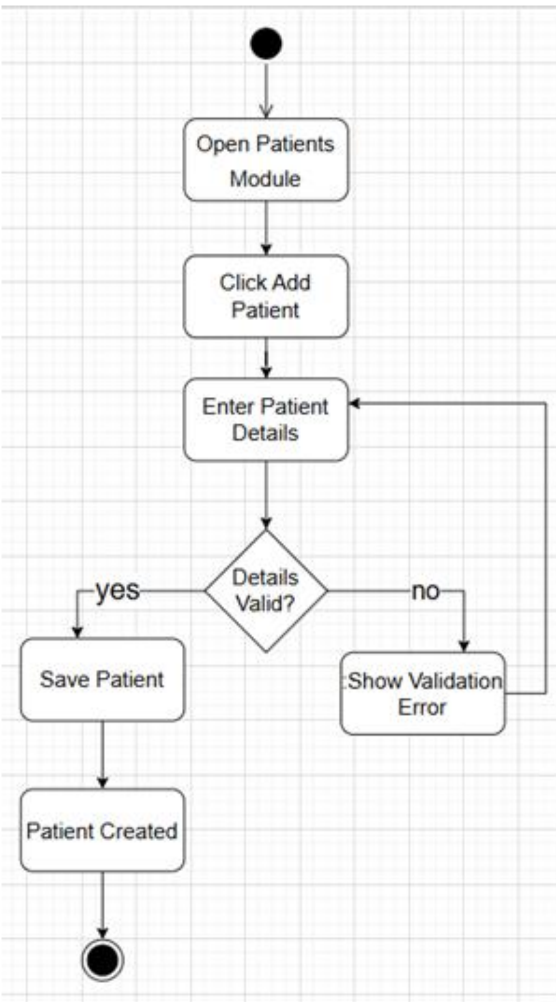


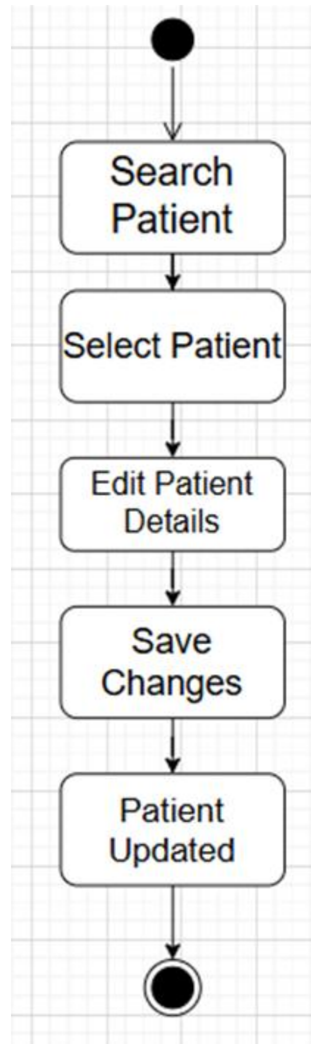

Open Staff Management
Click Add Doctor
Enter Doctor Details
Details Valid?
yes
no
Save Doctor
Show Validation Error
Doctor Created

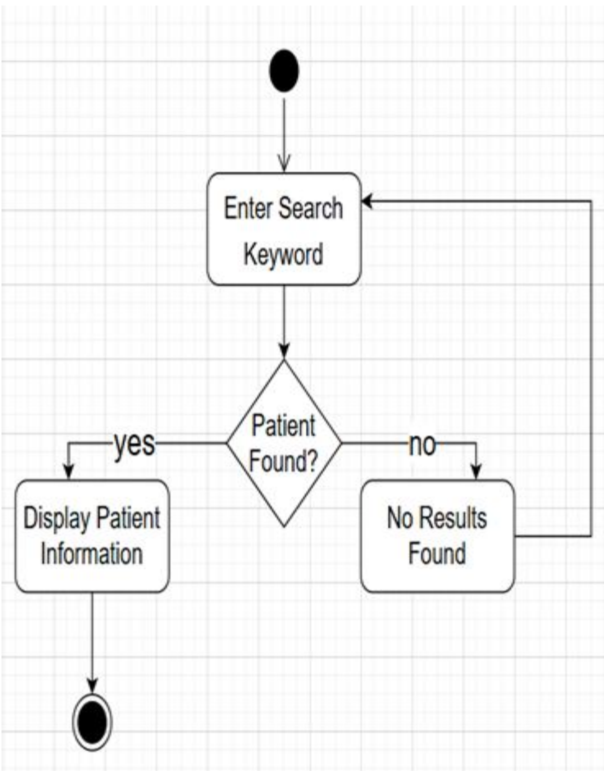

Open Staff Management
Click Add Receptionist
Enter Receptionist Details
Details Valid?
yes
no
Save Receptionist
Show Validation Error
Receptionist Created

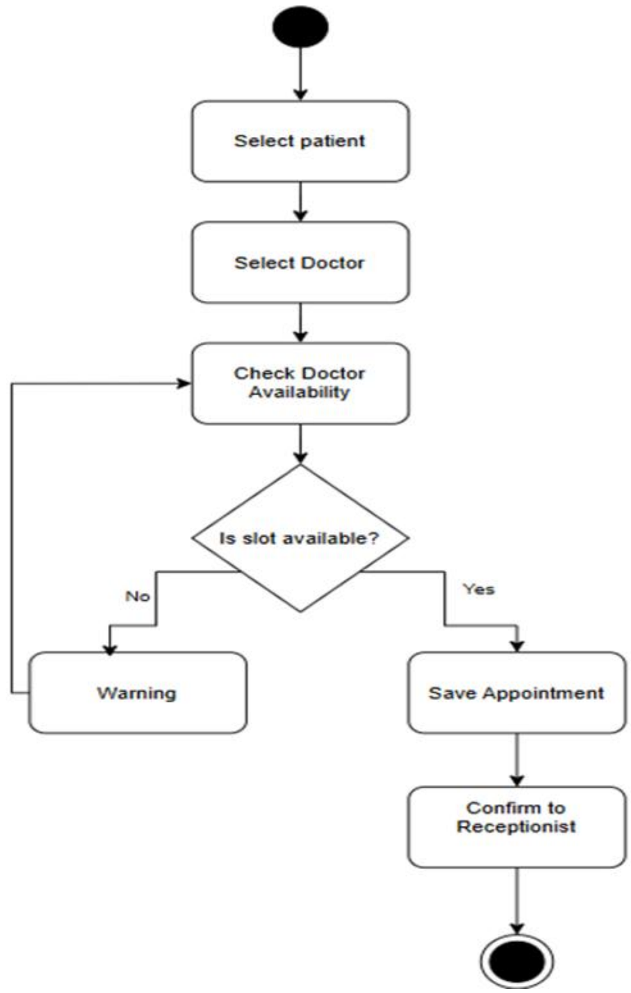

Open Doctors List
Select Doctor
Click Delete
Doctor Has Appointments?
yes
no
Show Warning
Confirm Deletion
Delete Doctor;

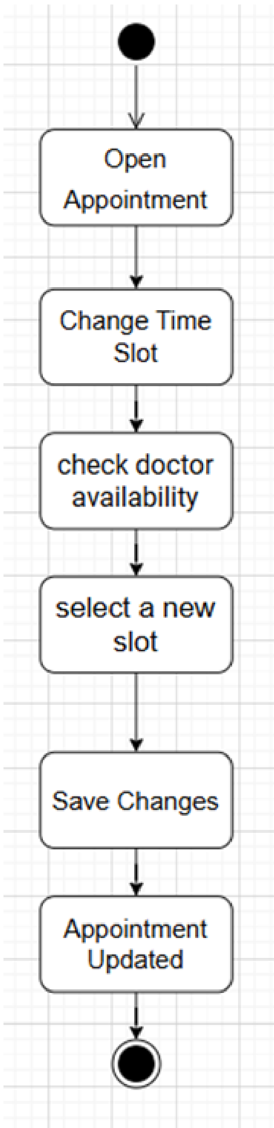

Open Receptionists List
Select Receptionist
Click Delete
Confirm Deletion
Receptionist Deleted

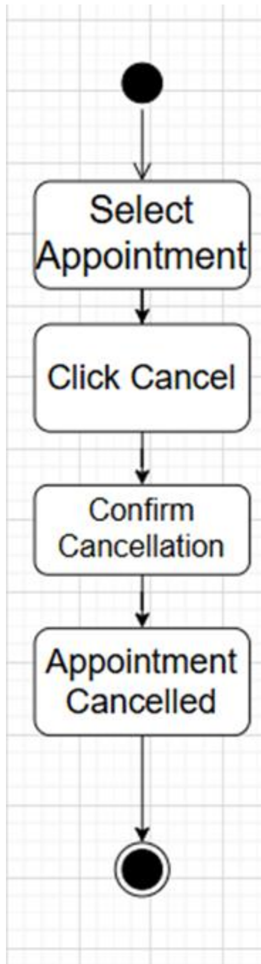

Open Patients Module
Click Add Patient
Enter Patient Details
Details Valid?
yes
no
Save Patient
Show Validation Error
Patient Created

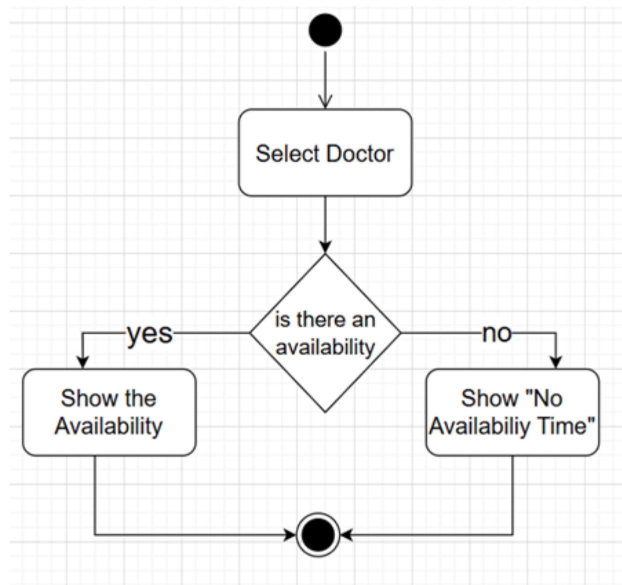

Search Patient
Select Patient
Edit Patient Details
Save Changes
Patient Updated

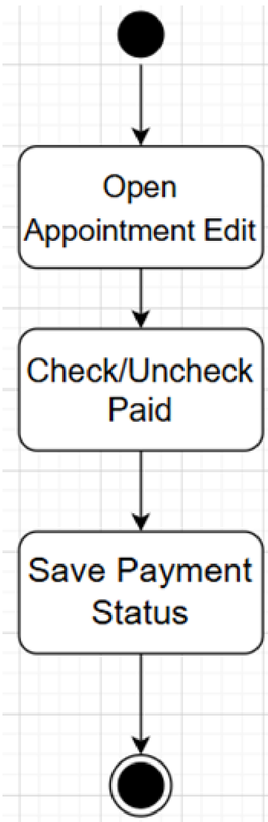

Enter Search Keyword
Patient Found?
yes
no
Display Patient Information
No Results Found

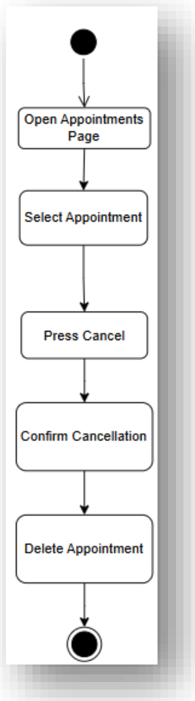

Select patient
Select Doctor
Check Doctor Availability
Is slot available?
No
Yes
Warning
Save Appointment
Confirm to Receptionist

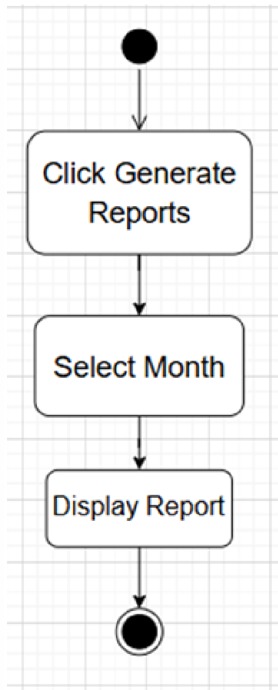

Open Appointment
Change Time Slot
check doctor availability
select a new slot
Save Changes
Appointment Updated

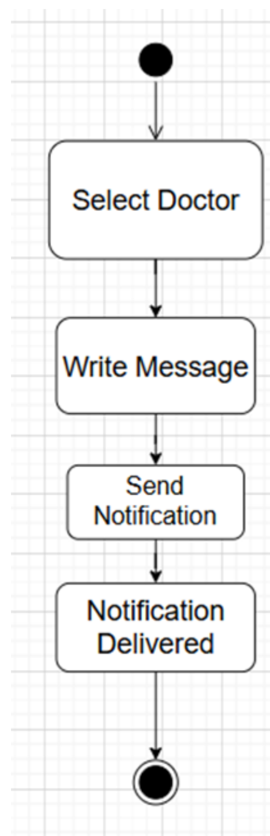

Select Appointment
Click Cancel
Confirm Cancellation
Appointment Cancelled

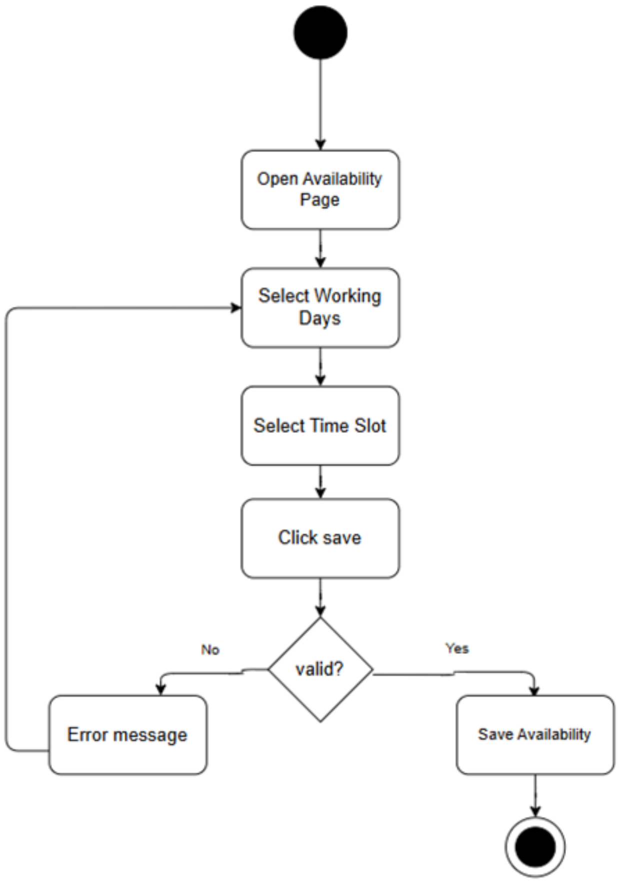

Select Doctor
is there an availability
yes
no
Show the Availability
Show "No Availabiliy Time"

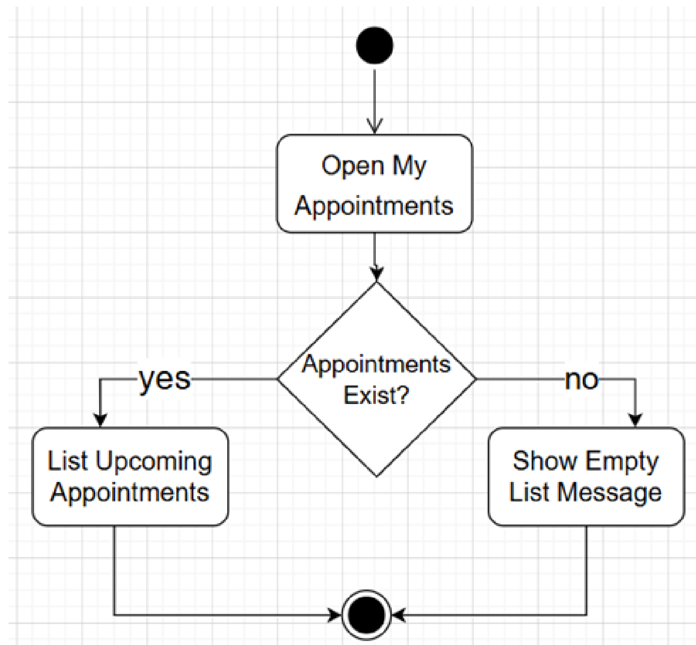

Open Appointment Edit
Check/Uncheck Paid
Save Payment Status

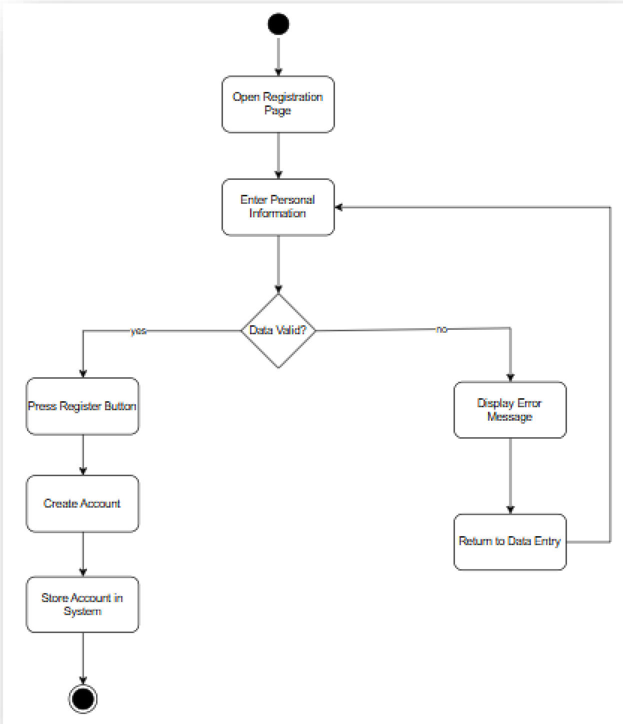

Open Appointments Page
Select Appointment
Press Cancel
Confirm Cancellation
Delete Appointment

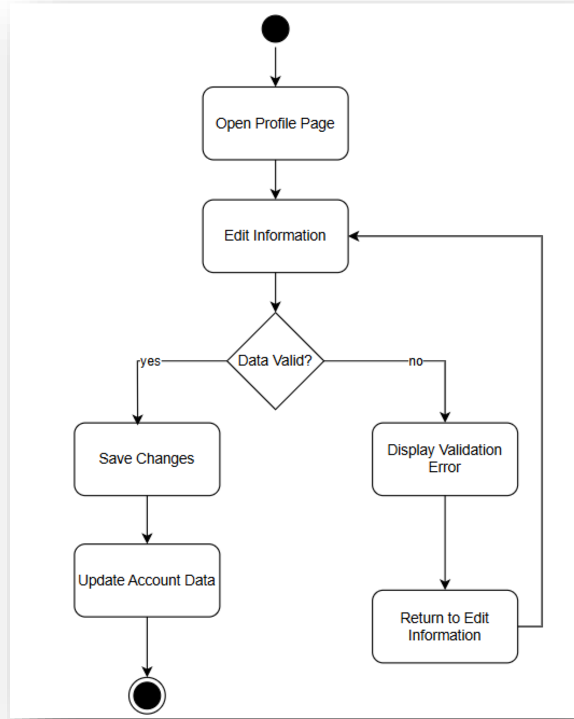

Click Generate Reports
Select Month
Display Report

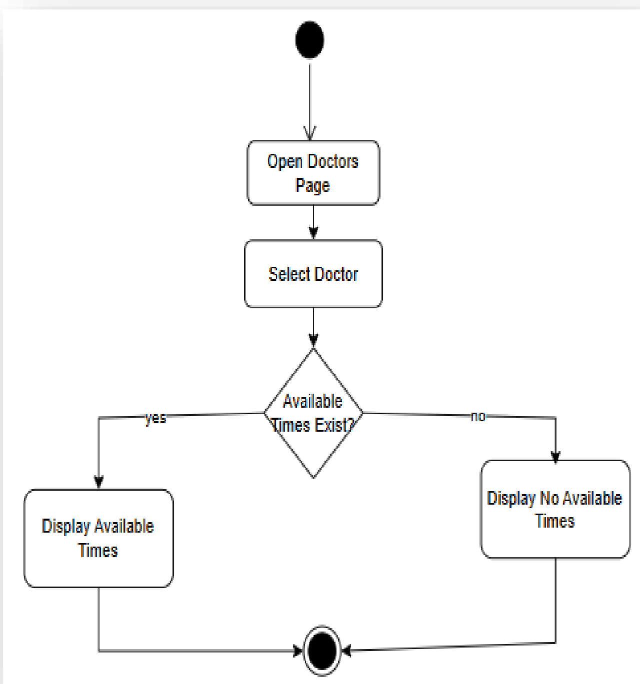

Select Doctor
Write Message
Send Notification
Notification Delivered

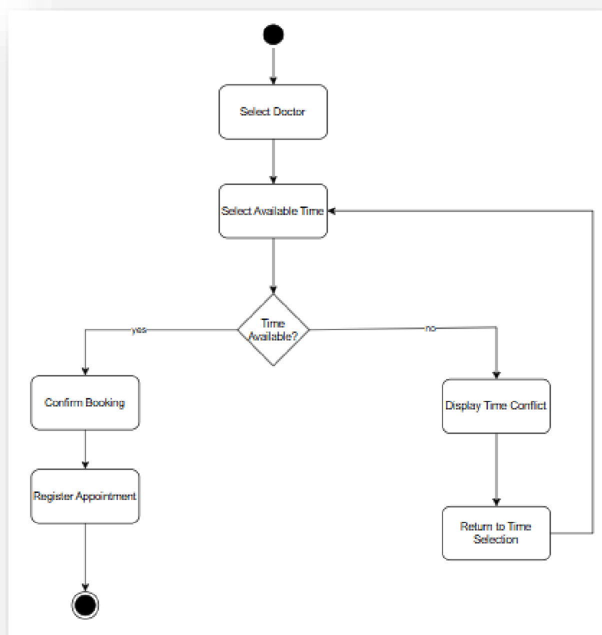

Open Availability Page
Select Working Days
Select Time Slot
Click save
valid?
No
Yes
Error message
Save Availability

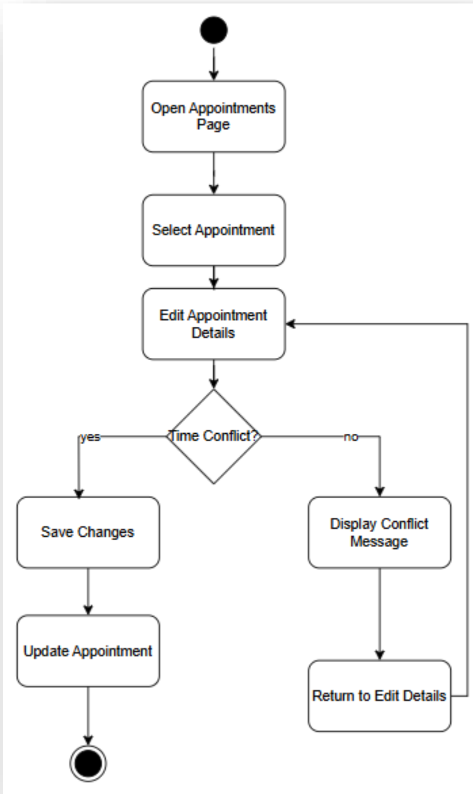

Open My Appointments
Appointments Exist?
yes
no
List Upcoming Appointments
Show Empty List Message

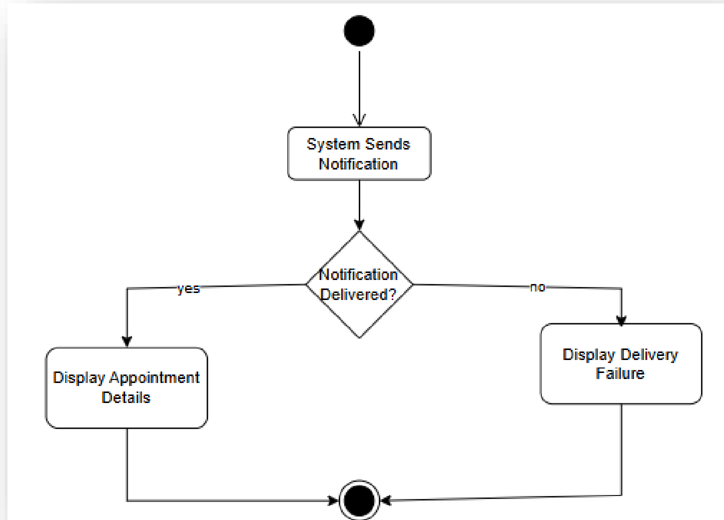

Open Registration Page
Enter Personal Information
Data Valid?
yes
no
Press Register Button
Create Account
Store Account in System
Display Error Message
Return to Data Entry

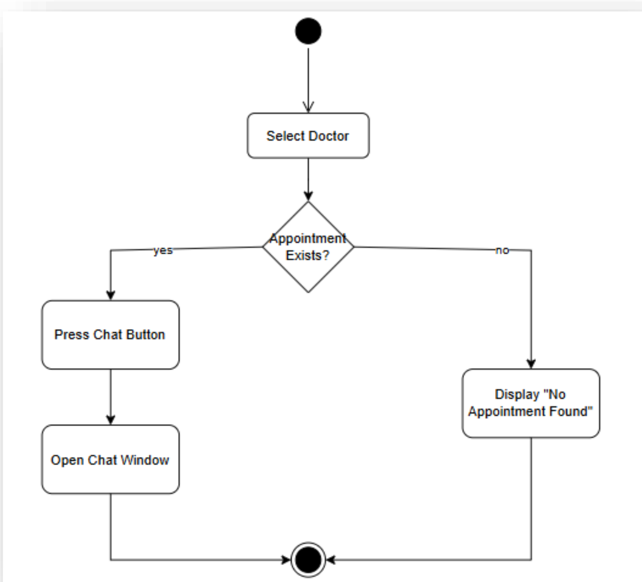

Open Profile Page
Edit Information
Data Valid?
yes
no
Save Changes
Update Account Data
Display Validation Error
Return to Edit Information

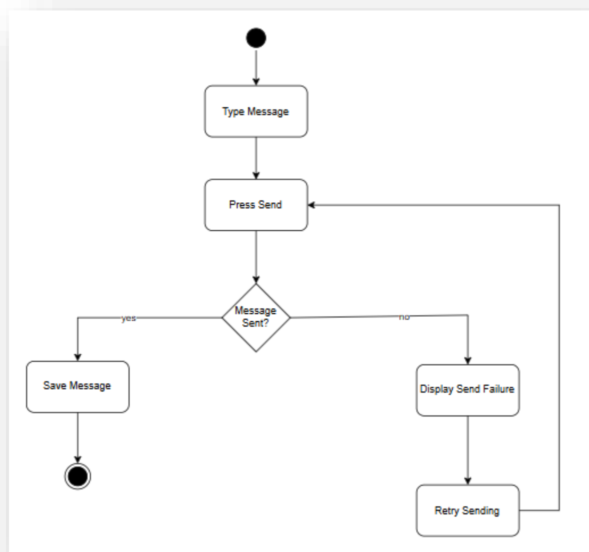

Open Doctors Page
Select Doctor
Available Times Exist?
yes
no
Display Available Times
Display No Available Times

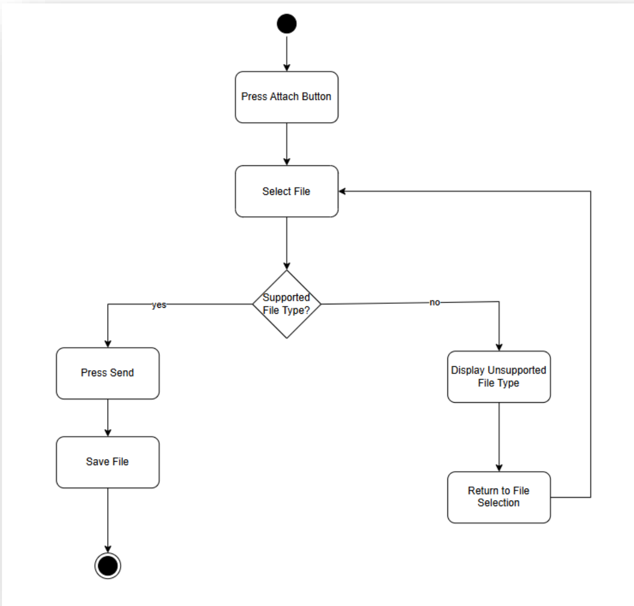

Select Doctor
Select Available Time
Time Available?
yes
no
Confirm Booking
Register Appointment
Display Time Conflict
Return to Time Selection

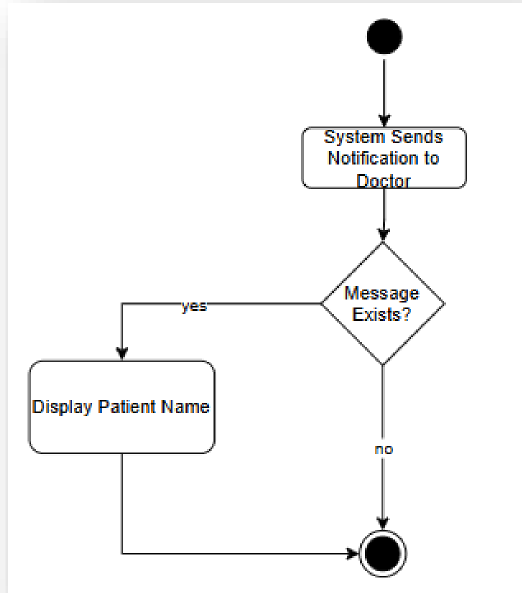

Open Appointments Page
Select Appointment
Edit Appointment Details
Time Conflict?
yes
no
Save Changes
Display Conflict Message
Update Appointment
Return to Edit Details

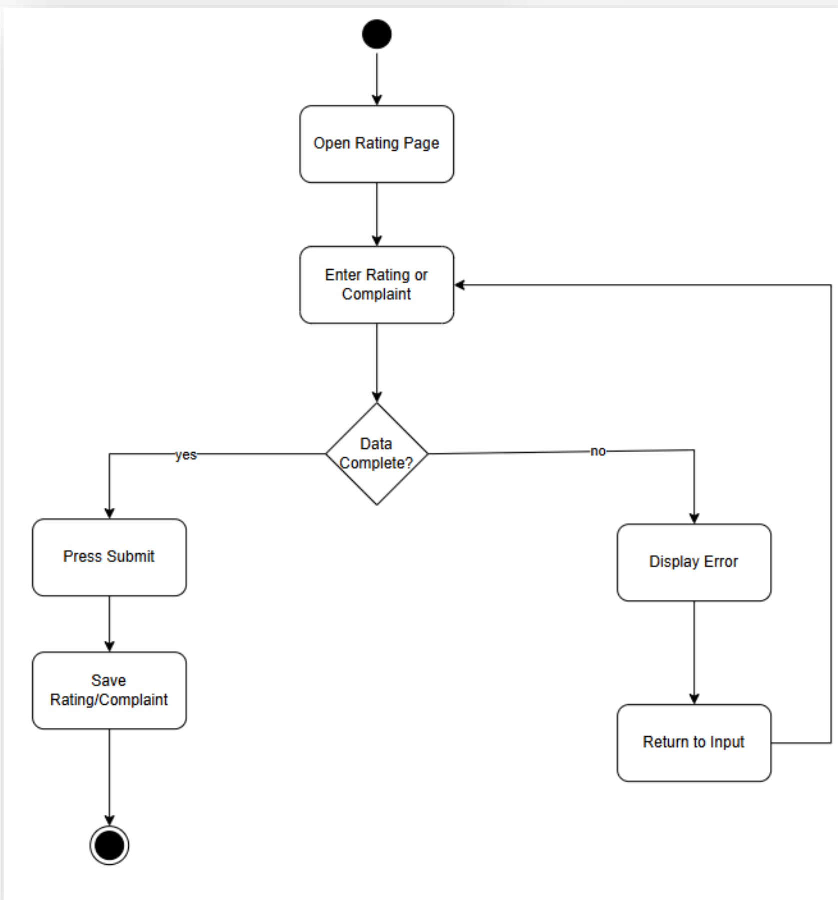

System Sends Notification
Notification Delivered?
yes
no
Display Appointment Details
Display Delivery Failure

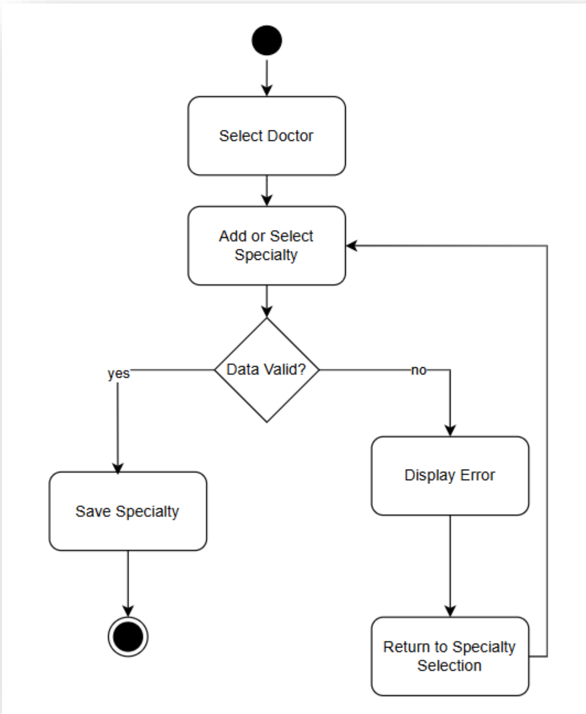

Select Doctor
Appointment Exists?
yes
no
Press Chat Button
Display "No Appointment Found"
Open Chat Window

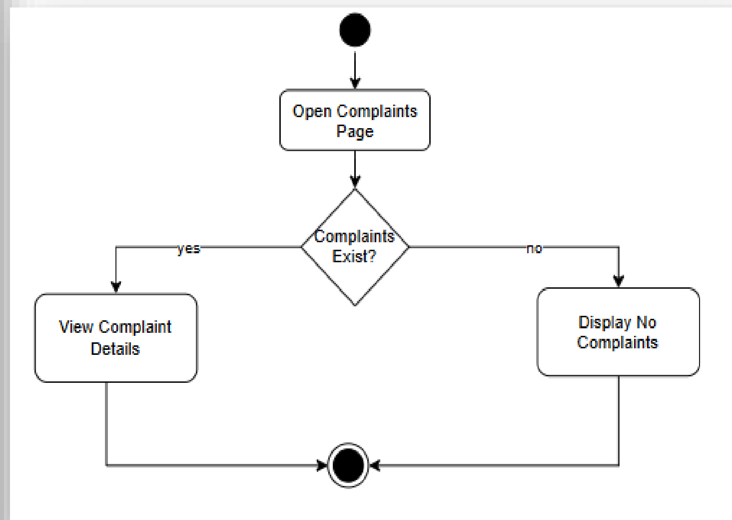

Type Message
Press Send
Message Sent?
yes
no
Save Message
Display Send Failure
Retry Sending

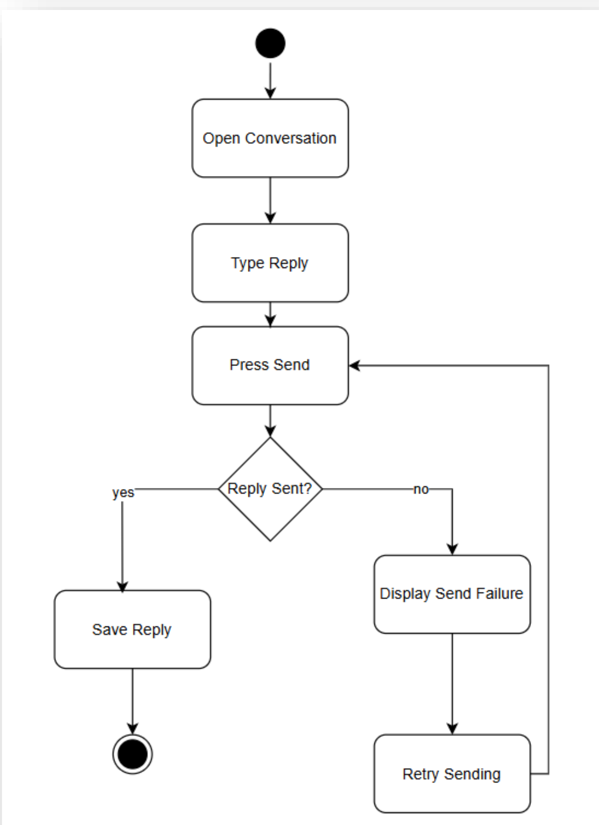

Press Attach Button
Select File
Supported File Type?
yes
no
Press Send
Save File
Display Unsupported File Type
Return to File Selection

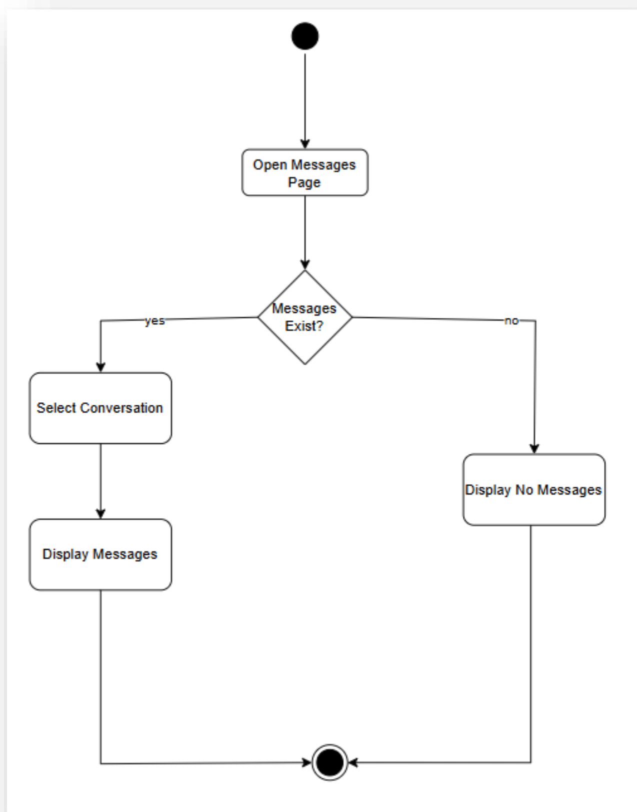

System Sends Notification to Doctor
Message Exists?
yes
no
Display Patient Name

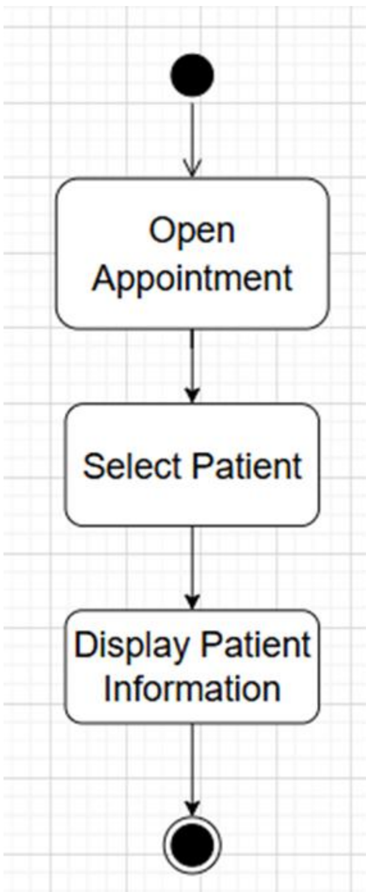

Open Rating Page
Enter Rating or Complaint
Data Complete?
yes
no
Press Submit
Save Rating/Complaint
Display Error
Return to Input

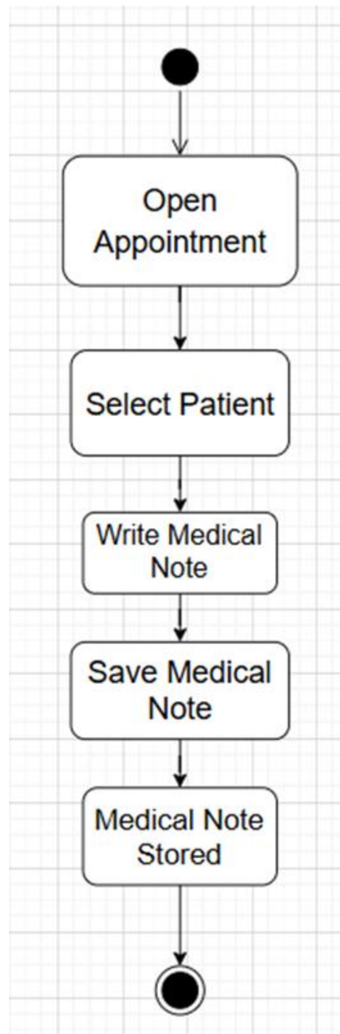

Select Doctor
Add or Select Specialty
Data Valid?
yes
no
Display Error
Save Specialty
Return to Specialty Selection


Open Complaints Page
Complaints Exist?
yes
no
View Complaint Details
Display No Complaints

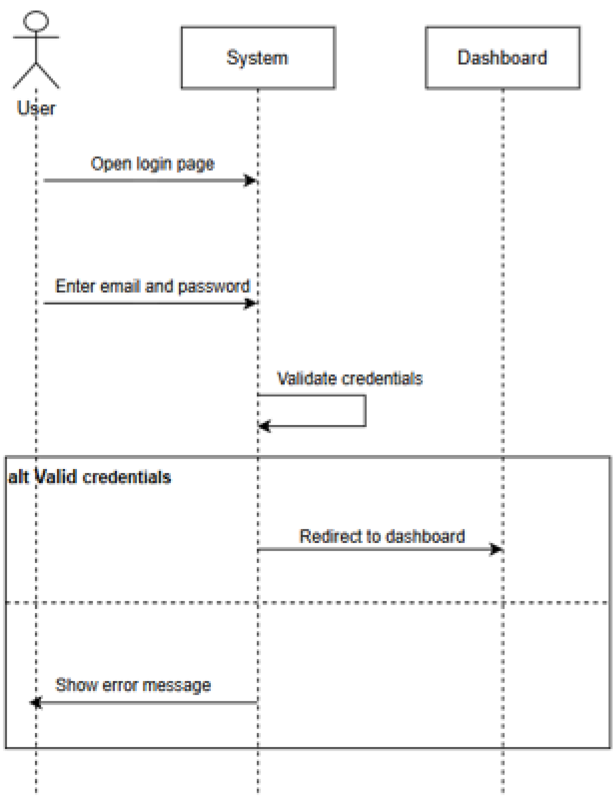

Open Conversation
Type Reply
Press Send
Reply Sent?
yes
no
Display Send Failure
Save Reply
Retry Sending

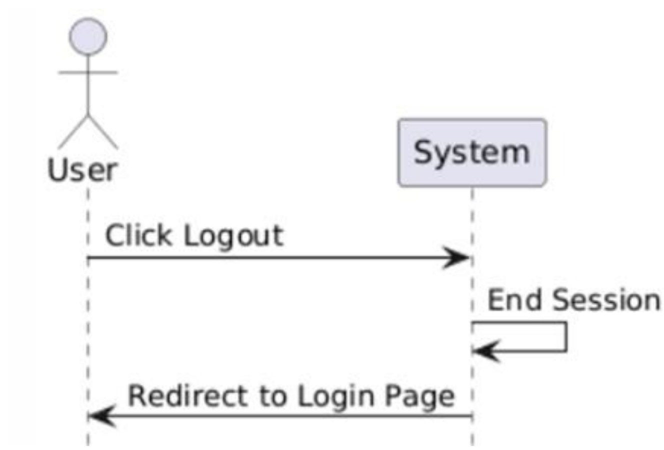

Open Messages Page
Messages Exist?
yes
no
Select Conversation
Display No Messages
Display Messages

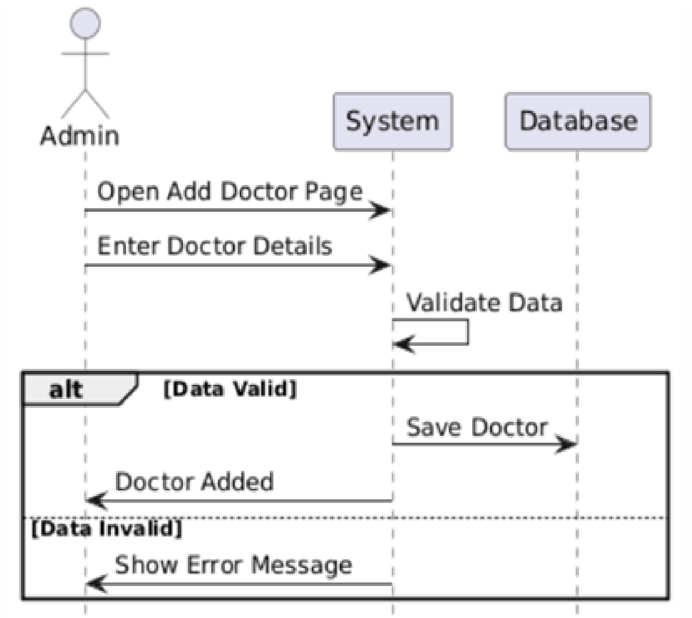

Open Appointment
Select Patient
Display Patient Information

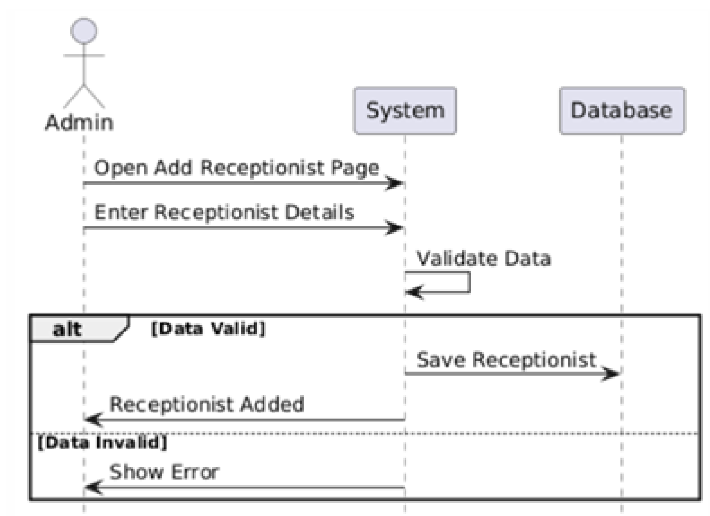

Open Appointment
Select Patient
Write Medical Note
Save Medical Note
Medical Note Stored

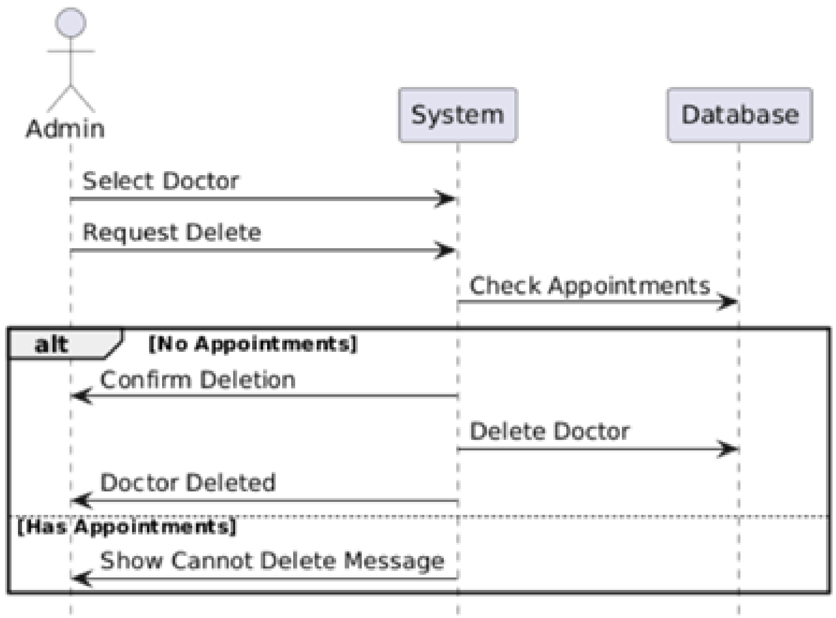
3.4.9 Sequence Diagram:

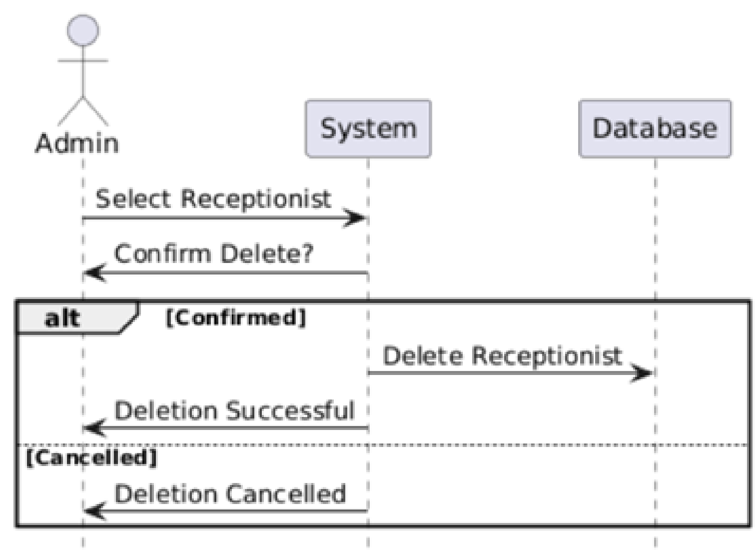


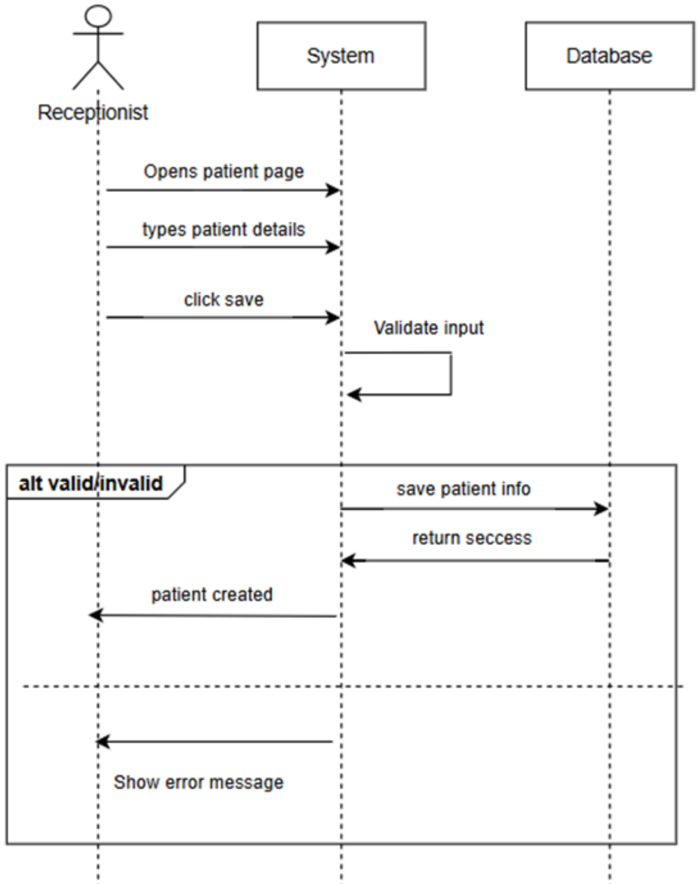



Admin
System
Database
Open Add Doctor Page
Enter Doctor Details
Validate Data
alt
[Data Valid]
Save Doctor
Doctor Added
[Data Invalid]
Show Error Message

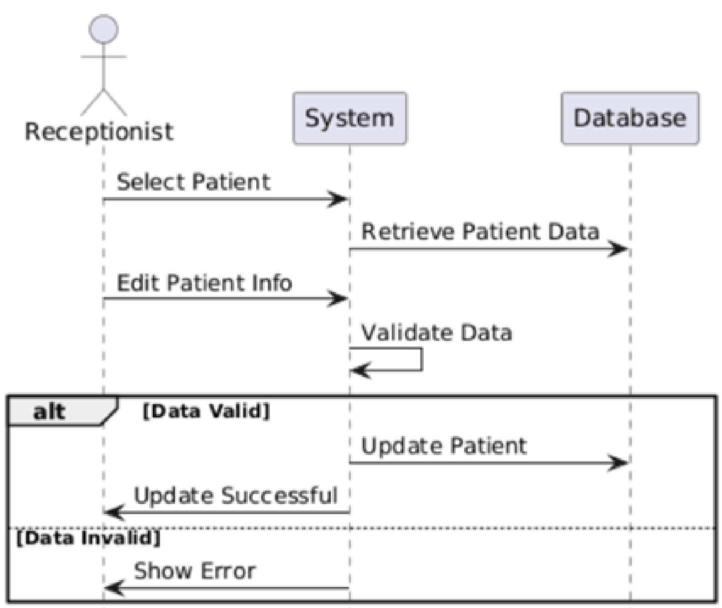

Admin
System
Database
Open Add Receptionist Page
Enter Receptionist Details
Validate Data
alt
[Data Valid]
Save Receptionist
Receptionist Added
[Data Invalid]
Show Error


Admin
System
Database
Select Doctor
Request Delete
Check Appointments
alt
[No Appointments]
Confirm Deletion
Delete Doctor
Doctor Deleted
[Has Appointments]
Show Cannot Delete Message

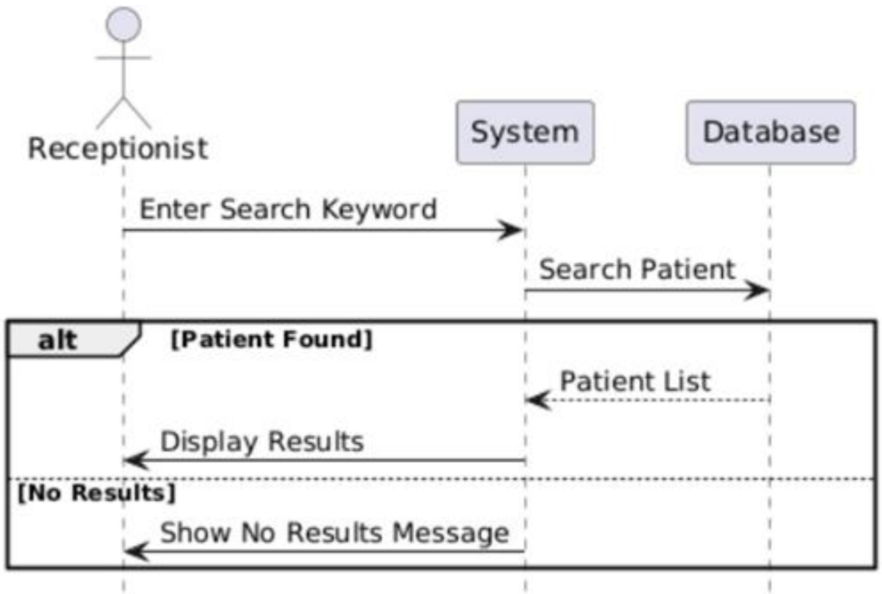

Admin
System
Database
Select Receptionist
Confirm Delete?
alt
[Confirmed]
Delete Receptionist
Deletion Successful
[Cancelled]
Deletion Cancelled

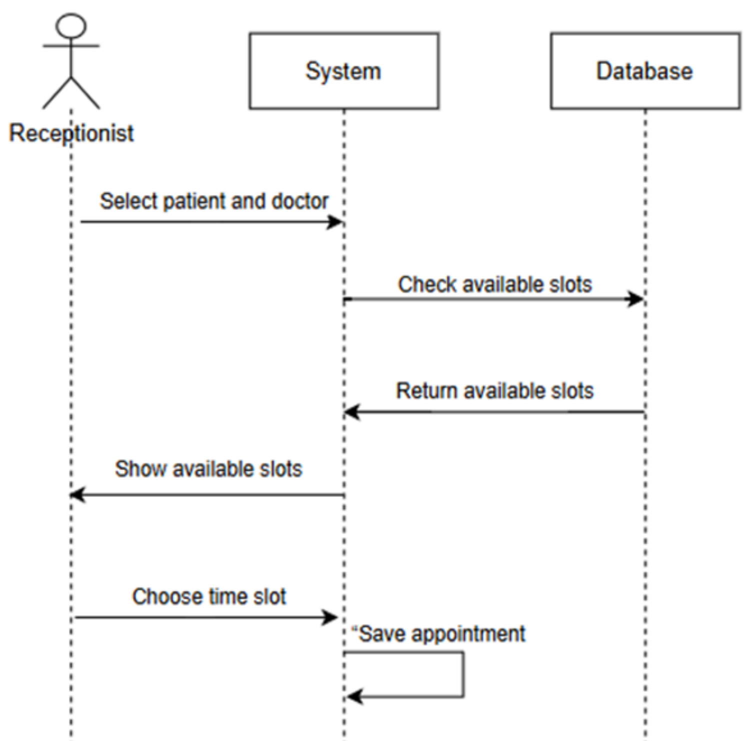


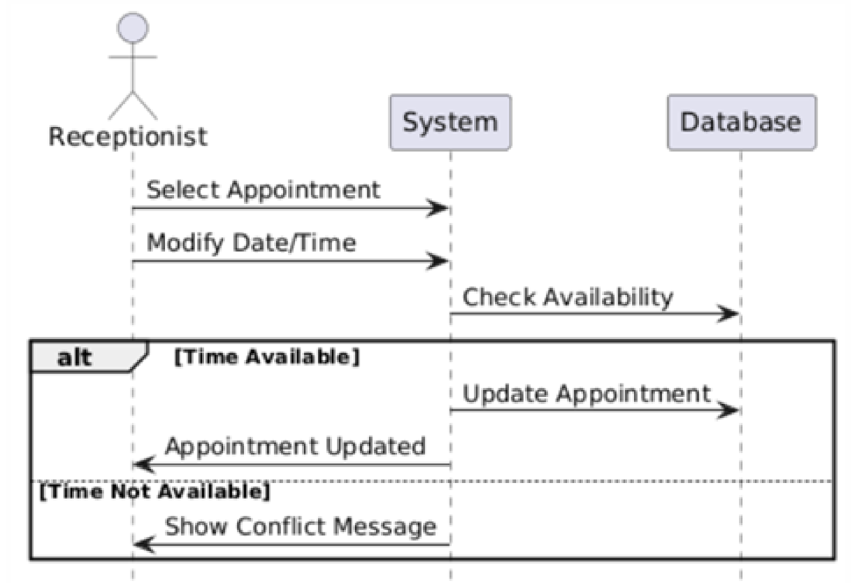
Receptionist
System
Database
Opens patient page
types patient details
click save
Validate input
alt valid/invalid
save patient info
return seccess
patient created
Show error message



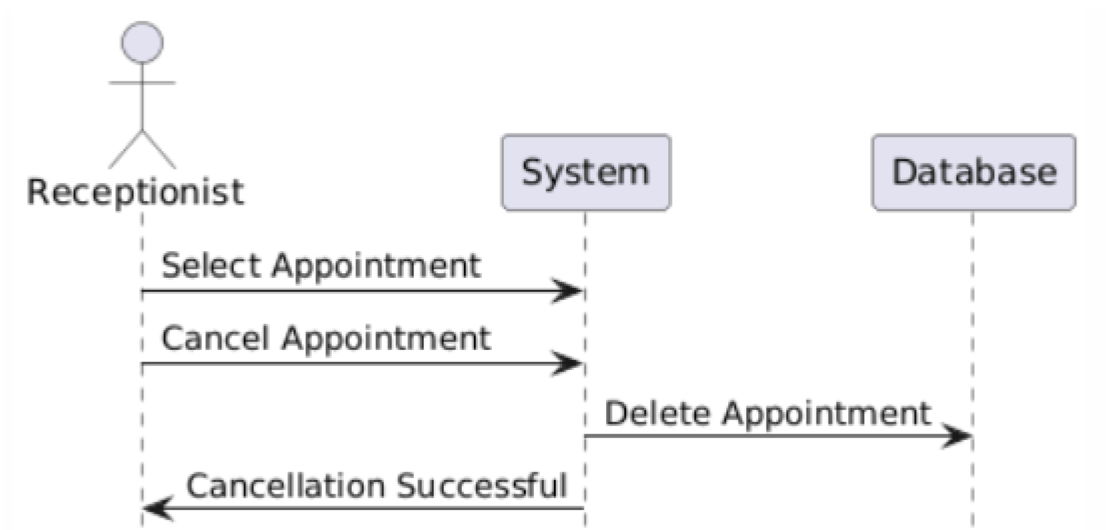
Receptionist
System
Database
Select Patient
Retrieve Patient Data
Edit Patient Info
Validate Data
alt
[Data Valid]
Update Patient
Update Successful
[Data Invalid]
Show Error

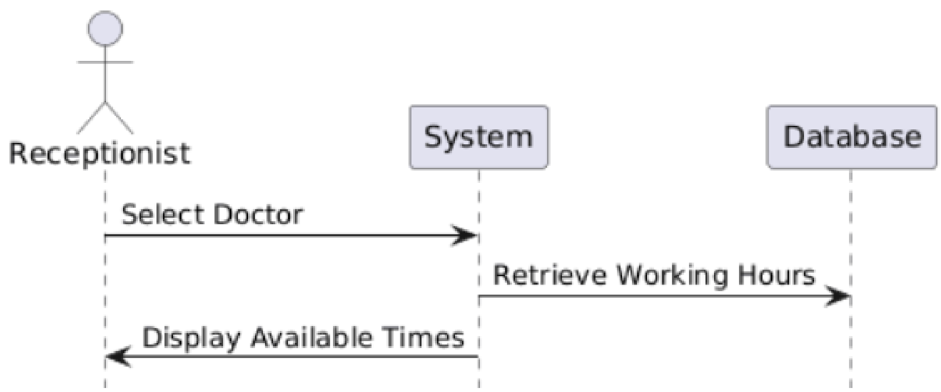

Receptionist
System
Database
Enter Search Keyword
Search Patient
alt
[Patient Found]
Patient List
Display Results
[No Results]
Show No Results Message

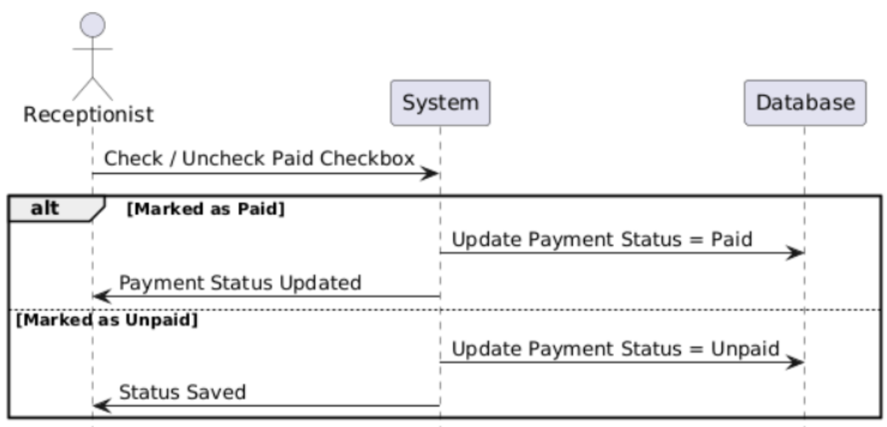

Receptionist
System
Database
Select patient and doctor
Check available slots
Return available slots
Show available slots
Choose time slot
"Save appointment

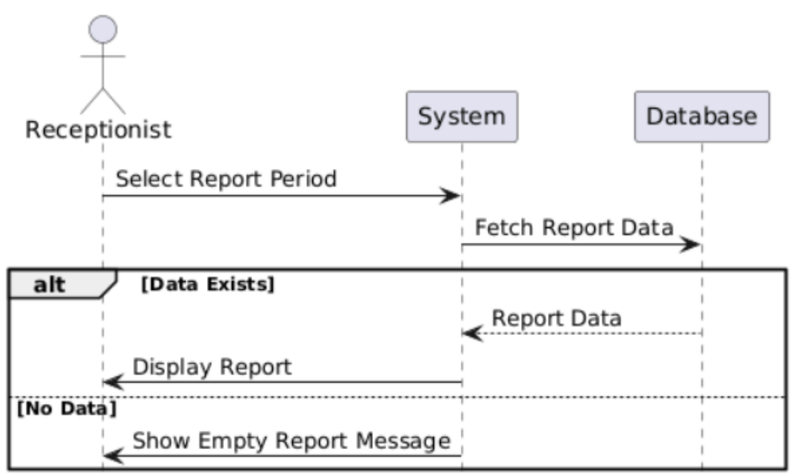

Receptionist
System
Database
Select Appointment
Modify Date/Time
Check Availability
alt
[Time Available]
Update Appointment
Appointment Updated
[Time Not Available]
Show Conflict Message

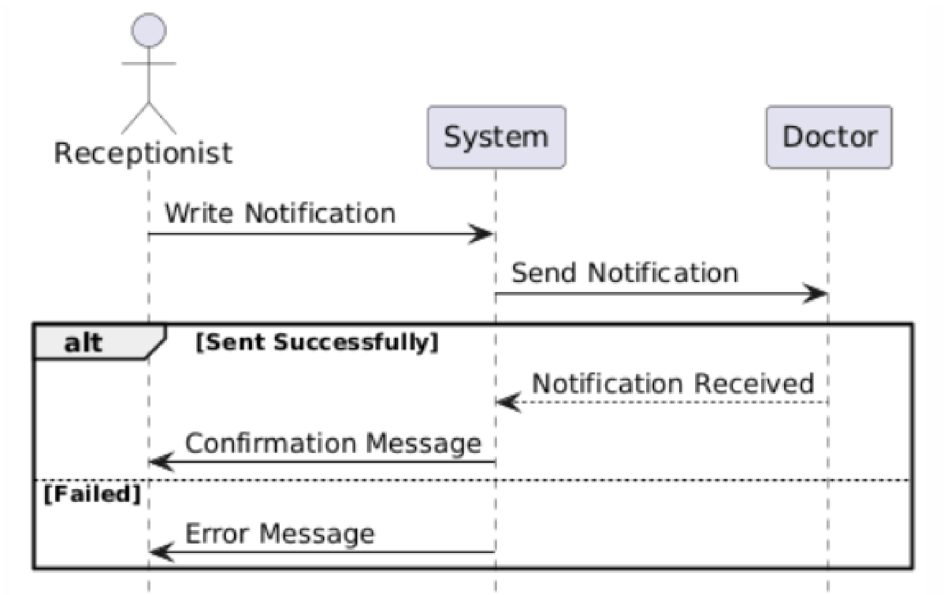

Doctor
System
NotificationService
Send Message Notification
Display Patient Name

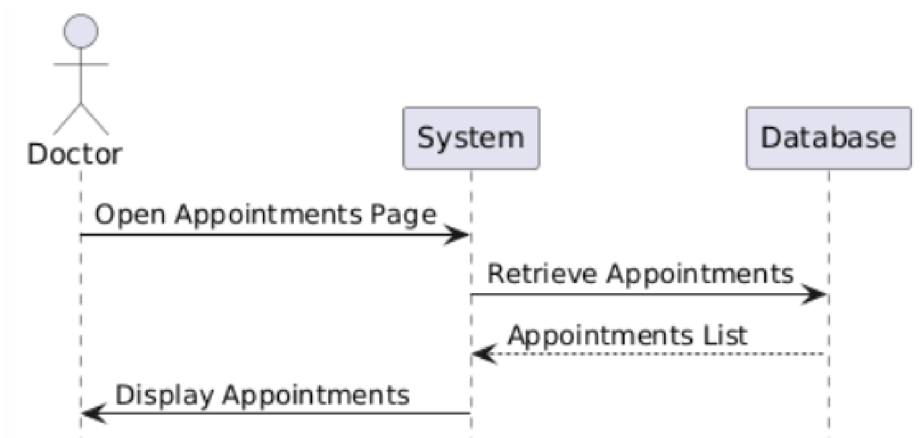

Receptionist
System
Database
Select Appointment
Cancel Appointment
Delete Appointment
Cancellation Successful

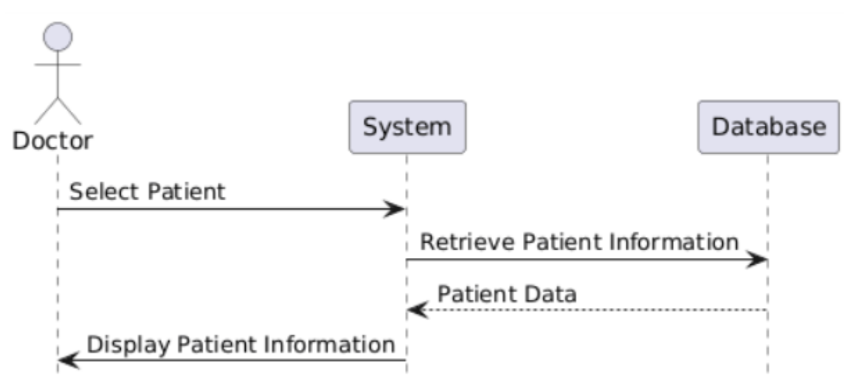

Receptionist
System
Database
Select Doctor
Retrieve Working Hours
Display Available Times

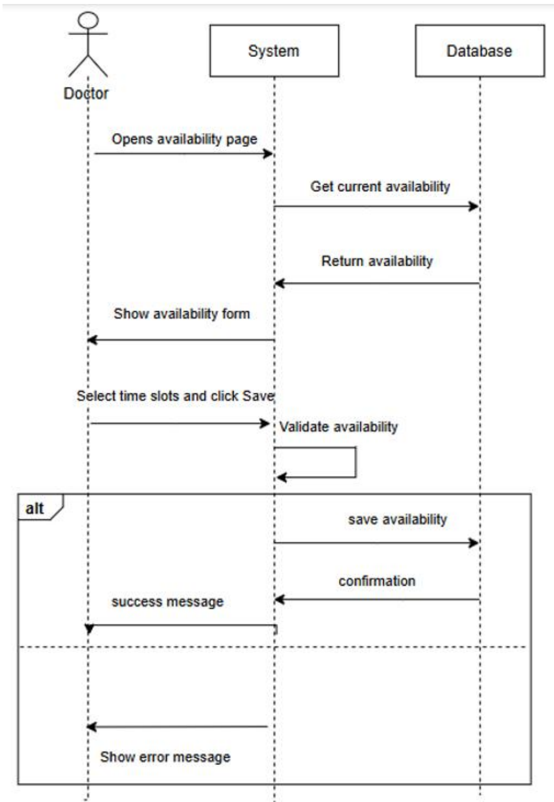

Receptionist
System
Database
Check / Uncheck Paid Checkbox
alt
[Marked as Paid]
Update Payment Status = Paid
Payment Status Updated
[Marked as Unpaid]
Update Payment Status = Unpaid
Status Saved

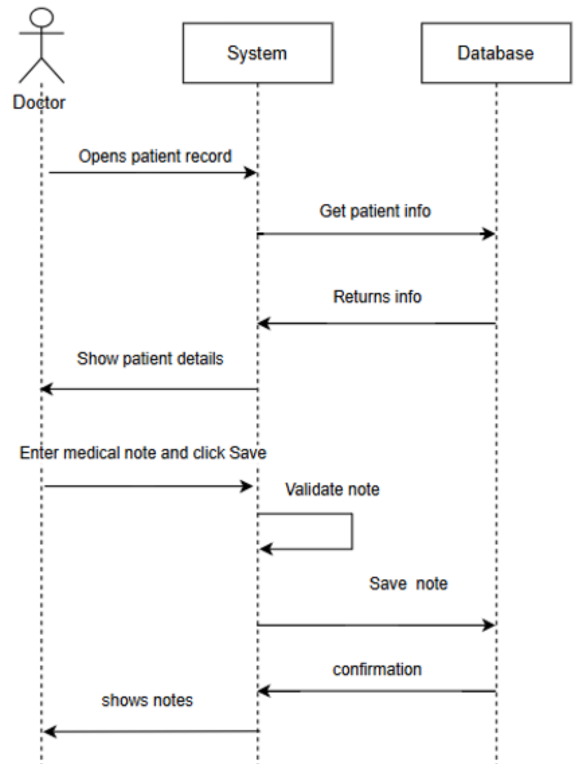

Receptionist
System
Database
Select Report Period
Fetch Report Data
alt
[Data Exists]
Report Data
Display Report
[No Data]
Show Empty Report Message

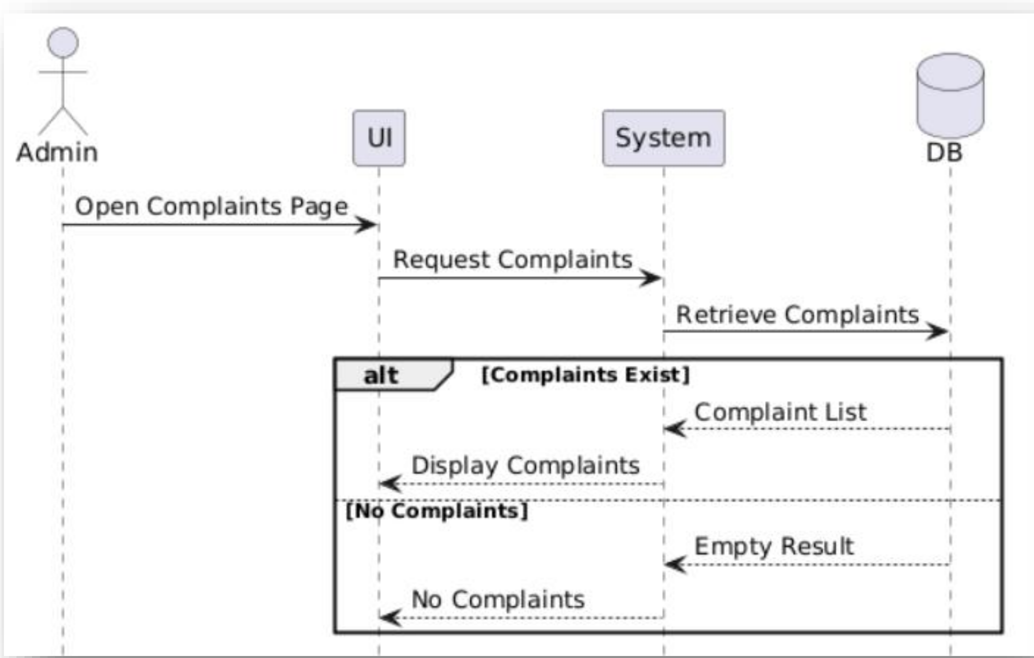

Receptionist
System
Doctor
Write Notification
Send Notification
alt
[Sent Successfully]
Notification Received
Confirmation Message
[Failed]
Error Message

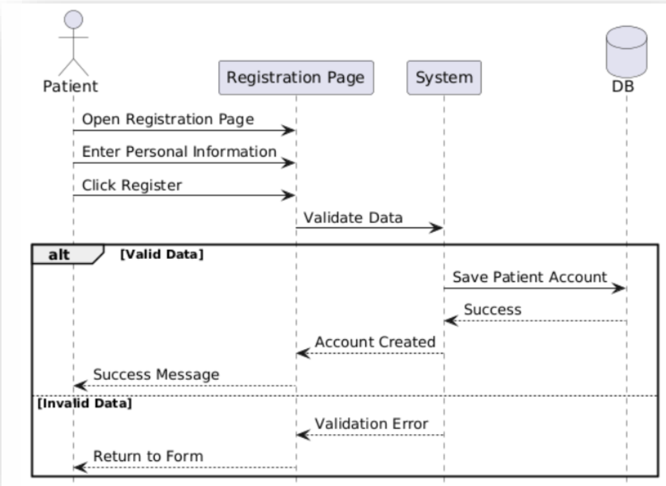

Doctor
System
Database
Open Appointments Page
Retrieve Appointments
Appointments List
Display Appointments

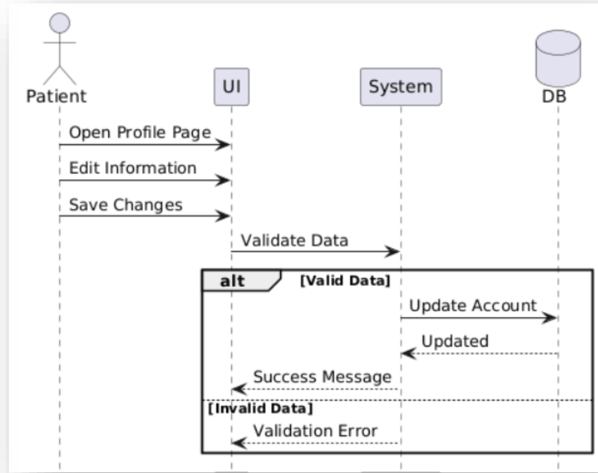

Doctor
System
Database
Select Patient
Retrieve Patient Information
Patient Data
Display Patient Information

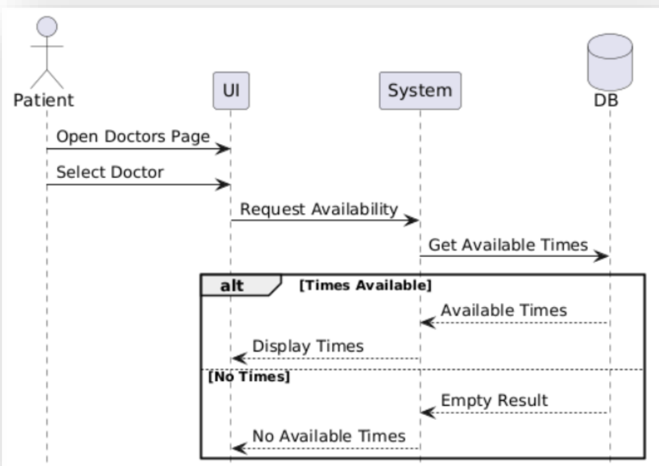

Doctor
System
Database
Opens availability page
Get current availability
Return availability
Show availability form
Select time slots and click Save
Validate availability
alt
save availability
confirmation
success message
Show error message

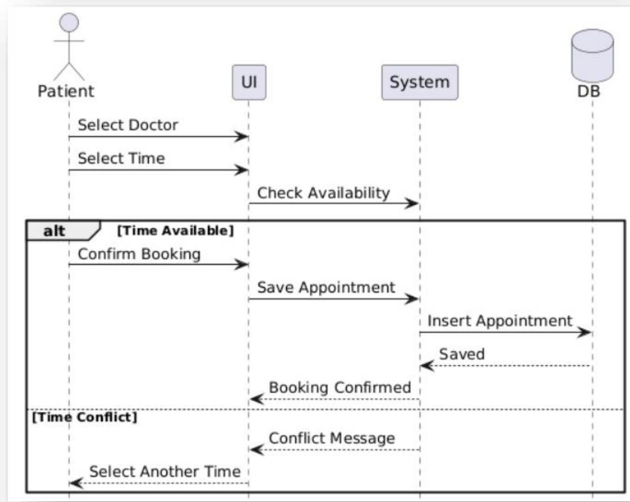

Doctor
System
Database
Opens patient record
Get patient info
Returns info
Show patient details
Enter medical note and click Save
Validate note
Save note
confirmation
shows notes

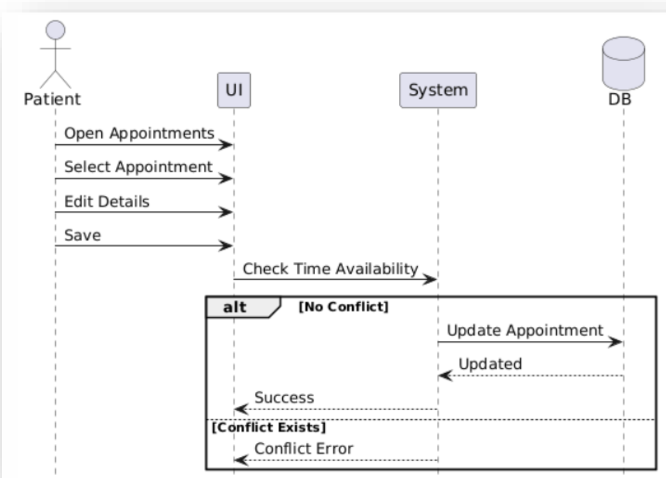

Admin
UI
System
DB
Open Complaints Page
Request Complaints
Retrieve Complaints
alt
[Complaints Exist]
Complaint List
Display Complaints
[No Complaints]
Empty Result
No Complaints

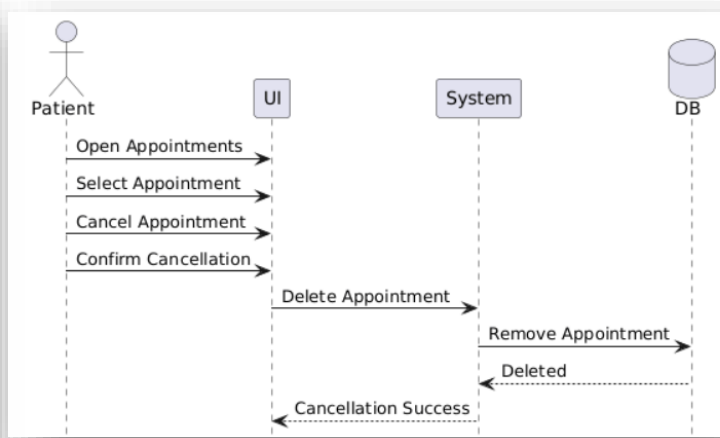

Patient
Registration Page
System
DB
Open Registration Page
Enter Personal Information
Click Register
Validate Data
alt
[Valid Data]
Save Patient Account
Success
Account Created
Success Message
[Invalid Data]
Validation Error
Return to Form

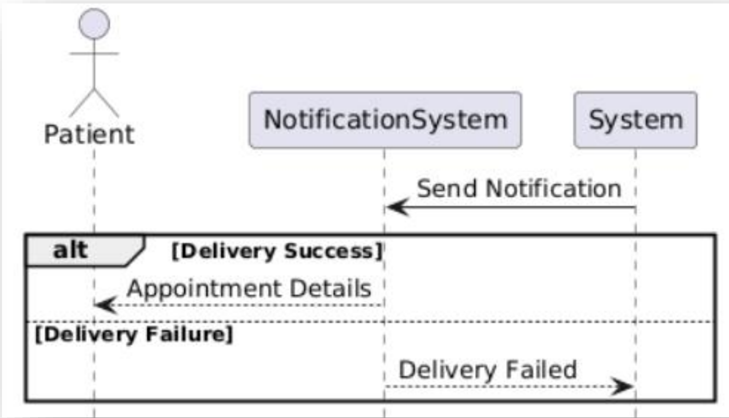

Patient
UI
System
DB
Open Profile Page
Edit Information
Save Changes
Validate Data
alt
[Valid Data]
Update Account
Updated
Success Message
[Invalid Data]
Validation Error


Patient
UI
System
DB
Open Doctors Page
Select Doctor
Request Availability
Get Available Times
alt
[Times Available]
Available Times
Display Times
[No Times]
Empty Result
No Available Times

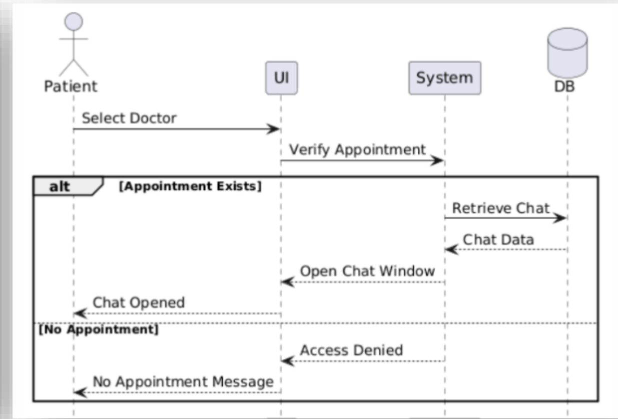

Patient
UI
System
DB
Select Doctor
Select Time
Check Availability
alt
[Time Available]
Confirm Booking
Save Appointment
Insert Appointment
Saved
Booking Confirmed
[Time Conflict]
Conflict Message
Select Another Time


Patient
UI
System
DB
Open Appointments
Select Appointment
Edit Details
Save
Check Time Availability
alt
[No Conflict]
Update Appointment
Updated
Success
[Conflict Exists]
Conflict Error

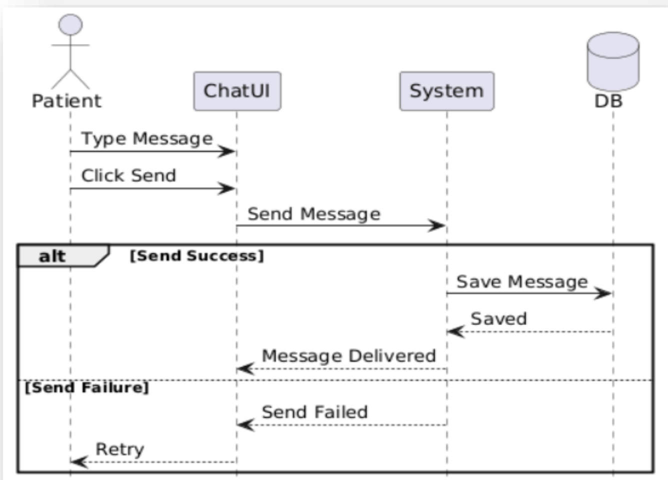

Patient
UI
System
DB
Open Appointments
Select Appointment
Cancel Appointment
Confirm Cancellation
Delete Appointment
Remove Appointment
Deleted
Cancellation Success



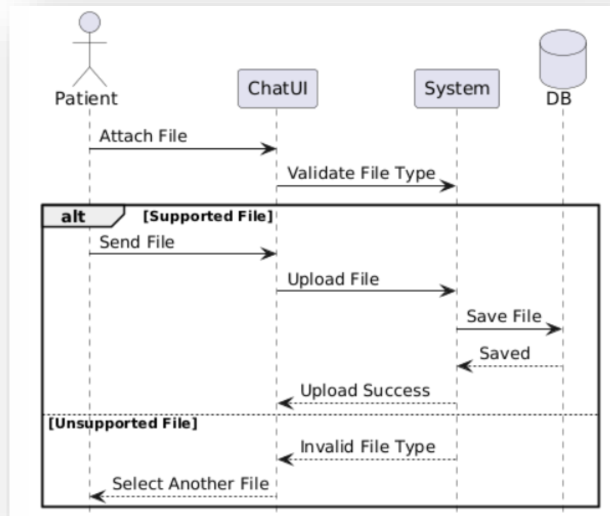
Patient
NotificationSystem
System
Send Notification
alt
[Delivery Success]
Appointment Details
[Delivery Failure]
Delivery Failed



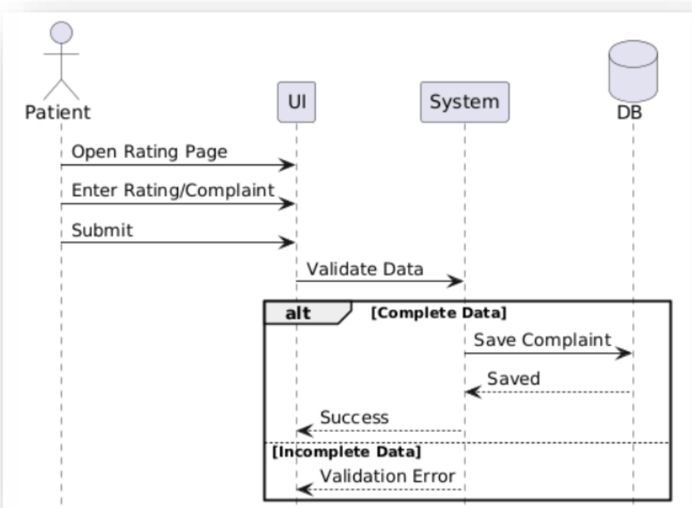
Patient
UI
System
DB
Select Doctor
Verify Appointment
alt
[Appointment Exists]
Retrieve Chat
Chat Data
Open Chat Window
Chat Opened
[No Appointment]
Access Denied
No Appointment Message



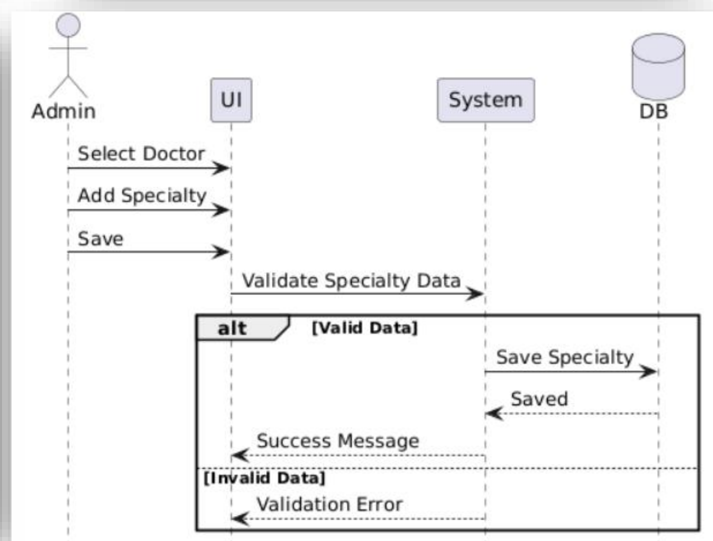
Patient
ChatUI
System
DB
Type Message
Click Send
Send Message
alt
[Send Success]
Save Message
Saved
Message Delivered
[Send Failure]
Send Failed
Retry



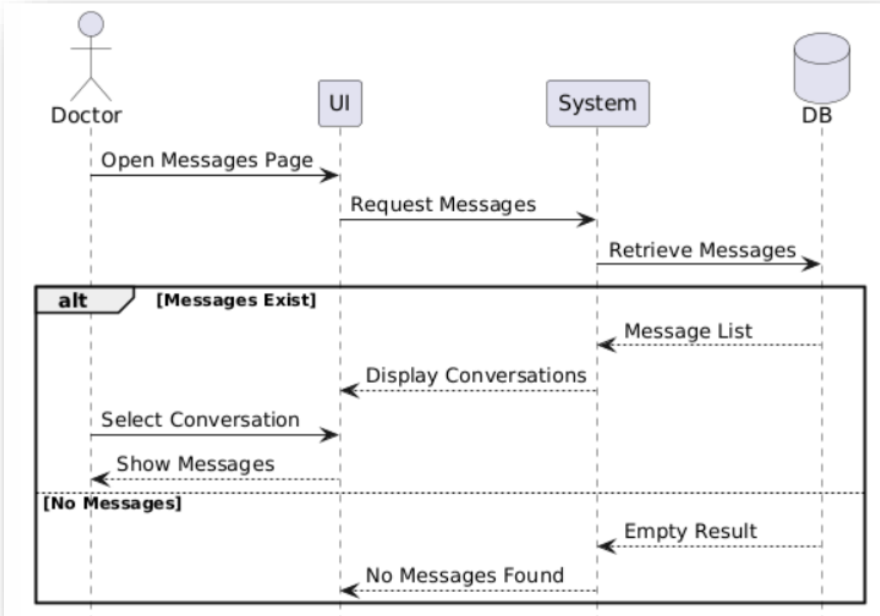
Patient
ChatUI
System
DB
Attach File
Validate File Type
alt
[Supported File]
Send File
Upload File
Save File
Saved
Upload Success
[Unsupported File]
Invalid File Type
Select Another File

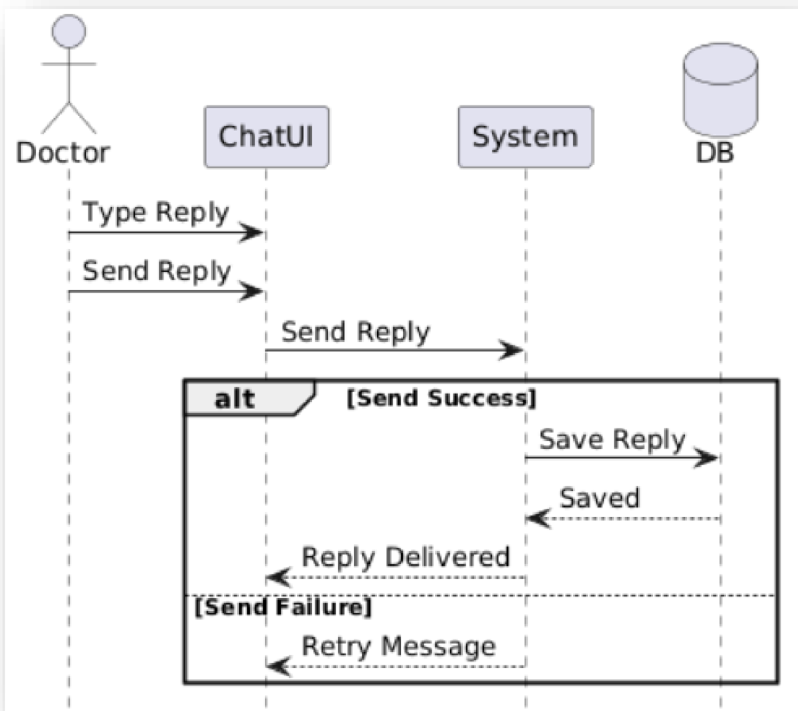


Patient
UI
System
DB
Open Rating Page
Enter Rating/Complaint
Submit
Validate Data
alt
[Complete Data]
Save Complaint
Saved
Success
[Incomplete Data]
Validation Error



Admin
UI
System
DB
Select Doctor
Add Specialty
Save
Validate Specialty Data
alt
[Valid Data]
Save Specialty
Saved
Success Message
[Invalid Data]
Validation Error


Doctor
UI
System
DB
Open Messages Page
Request Messages
Retrieve Messages
alt
[Messages Exist]
Message List
Display Conversations
Select Conversation
Show Messages
[No Messages]
Empty Result
No Messages Found


Doctor
ChatUI
System
DB
Type Reply
Send Reply
Send Reply
alt
[Send Success]
Save Reply
Saved
Reply Delivered
[Send Failure]
Retry Message

3.4.10 Unit Testing Schedule:

Test ID	Feature / FR	Preconditions	Test Steps	Expected Result	Actual Result
TC-01	Login (FR-1)	User exists	1) Open login 2) Enter valid email and password 3) Click Login	Redirect to correct dashboard based on role	As expected redirected to correct dashboard
TC-02	Login invalid (FR-1)	None	1) Open login 2) Enter wrong password 3) Click Login	Error message shown, user stays on login	As expected (error shown, stayed on login)
TC-03	Logout (FR-2)	Logged in	1) Click Logout	Session ends, redirected to login	As expected (logged out and returned to login)
TC-04	Add Doctor (FR-3)	Admin logged in	1) Staff page 2) Add Doctor 3) Fill fields 4) Save	Doctor user created with role doctor	As expected (doctor created and listed)
TC-05	Add Doctor duplicate email (FR-3)	Admin logged in	1) Add Doctor 2) Use existing email 3) Save	Validation error, no user created	As expected (validation error shown, no new user)
TC-06	Delete Doctor no appointments (FR-4)	Admin logged in, doctor exists	1) Select doctor 2) Delete 3) Confirm	Doctor removed	As expected (doctor removed successfully)
TC-07	Delete Doctor with appointments (FR-4)	Admin logged in, doctor has appointments	1) Select doctor 2) Delete	System shows message	As expected (message shown)
TC-08	Add Receptionist (FR-5)	Admin logged in	1) Staff page 2)Add Receptionist 3) Save	Receptionist account created	As expected (receptionist created and listed)
TC-09	Delete Receptionist (FR-6)	Admin logged in, receptionist exists	1) Select receptionist 2) Delete	Receptionist removed	As expected (receptionist removed successfully)
TC-10	Add Patient (FR-7)	Receptionist logged in	1) Patients 2) Add Patient 3) Fill required fields 4) Save	Patient record saved and appears in list	As expected (patient saved and appears in list)
TC-11	Edit Patient (FR-8)	Receptionist logged in, patient exists	1) Open patient 2) Edit fields 3) Save	Updated info saved	As expected (patient info updated)
TC-12	Search Patient (FR-9)	Receptionist logged in, patient exists	1) Search by keyword	Matching patient shown	As expected (matching patient returned)
TC-13	Search Patient not found (FR-9)	Receptionist logged in	1) Search random text	“No results” / empty list shown	As expected (no results shown)
TC-14	View Doctor Availability (FR-13)	Receptionist logged in, doctor has availability	1) Choose doctor + date 2) Load slots	Available slots displayed	As expected (available slots displayed)
TC-15	Book Appointment success (FR-10)	Receptionist logged in and patient and doctor exist with slot free	1) Create appointment 2) Pick doctor and date and time 3) Save	Appointment created	As expected (appointment created)
TC-16	Book Appointment slot conflict (FR-10)	Slot already booked	1) Try booking same doctor/date/time	System rejects with warning/error	As expected (conflict blocked with message)

TC-17	Modify Appointment success (FR-11)	Appointment exists	1) Edit appointment 2) Change time to free slot 3) Save	Appointment updated	As expected (appointment updated)
TC-18	Modify Appointment to taken slot (FR-11)	Another appointment exists at target slot	1) Edit appointment 2) Choose taken slot 3) Save	System rejects change	As expected (update blocked with message)
TC-19	Cancel Appointment (FR-12)	Appointment exists	1) Cancel appointment 2) Confirm	Status becomes cancelled, removed from active list (if filtered)	As expected (status changed to cancelled)
TC-20	Mark Paid/Unpaid (FR-14)	Appointment exists	1) Edit appointment 2)Toggle Paid checkbox 3) Save	is_paid updates correctly and shows on details/report	As expected (paid/unpaid updated correctly)
TC-21	Monthly Report (FR-15)	Receptionist logged in, some appointments exist	1) Press generate reports	Shows reports	As expected (report shows correct counts/list)
TC-22	Doctor View Appointments (FR-17)	Doctor logged in	1) Open My Appointments	Doctor sees only their appointments	As expected (doctor sees own appointments only)
TC-23	Doctor View Patient Info (FR-18)	Doctor logged in, has appointment	1) Open appointment 2) View patient info	Patient info shown	As expected (patient info displayed)
TC-24	Set Availability valid (FR-19)	Doctor logged in	1) Open availability 2) choose time slots 3) Save	Availability saved and visible	As expected (availability saved)
TC-25	Add Medical Note valid (FR-20)	Doctor logged in, patient exists	1) Open patient appointment 2) Add note 3) Save	Note saved and shown	As expected (note saved and displayed)
TC-26	Add Medical Note empty (FR-20)	Doctor logged in	1) Submit empty note	Error message shown	As expected (error shown)
TC-27	Role access control (Security)	Logged in as receptionist	1) Try opening admin staff URL	Access denied / redirected	As expected (access blocked)
TC-28	Role access control (Security)	Logged in as doctor	1) Try opening receptionist patients URL	Access denied / redirected	As expected (access blocked)
TC-29	Create a new patient account (FR-21)	Email hasn't been used before	1) Go to the registration page 2) Fill in name, email, password and phone number 3) Click Register	The account is created and the user gets logged in right away	Worked as expected the account was created and got redirected to the patient dashboard
TC-30	Register with invalid info (FR-21)	None	1) Open the registration page 2) Use an email that already exists, or leave a required field blank 3) Click Register	An error message shows up and the account is not created	Worked as expected the validation message was shown
TC-31	Update patient profile info (FR-22)	Patient is signed in	1) Open the profile page 2) Change the name or phone number 3) Hit Save	The new details are saved with no issues	Worked as expected the info got updated

TC-32	Save profile with bad data (FR-22)	Patient is signed in	1) Open the profile page 2) Type an email in the wrong format 3) Hit Save	An error message shows and nothing gets saved	Worked as expected the validation message appeared
TC-33	Check a doctor's available times (FR-23)	Patient is signed in and the doctor has added availability	1) Open the doctors page 2) Pick a doctor 3) Look at their schedule	The doctor's free time slots are listed	Worked as expected the available times were shown
TC-34	Check availability for a doctor with no open slots (FR-23)	Patient is signed in	1) Open the doctors page 2) Pick a doctor who hasn't added any availability	A "No available times" message appears	Worked as expected the "no available times" message showed
TC-35	Book an appointment (FR-24)	Patient is signed in and there's an open slot	1) Pick a doctor 2) Choose one of the open times 3) Confirm the booking	The appointment gets saved and shows up under My Appointments	Worked as expected the appointment was booked
TC-36	Try to book a slot someone else already took (FR-24)	The time is already taken	1) Try to book a time that's already been reserved	A conflict message appears and the booking is blocked	Worked as expected the booking was rejected with a message
TC-37	Reschedule an appointment (FR-25)	The appointment exists	1) Open My Appointments 2) Click the appointment 3) Pick a different open time 4) Save	The appointment shows the new time	Worked as expected the appointment was updated
TC-38	Reschedule to a time that's already booked (FR-25)	That time is already reserved	1) Open My Appointments 2) Click the appointment 3) Pick a time that's taken 4) Save	A conflict message shows and the change isn't saved	Worked as expected the edit was rejected
TC-39	Cancel an appointment (FR-26)	The appointment exists	1) Open My Appointments 2) Click the appointment 3) Hit Cancel 4) Confirm	The appointment is marked as cancelled	Worked as expected the appointment was cancelled
TC-40	Appointment notification (FR-27)	The patient has an appointment	1) Book or change an appointment 2) Open the notifications page	A notification with the appointment details appears	Worked as expected the notification showed up
TC-41	Start a chat with a doctor (FR-28)	Patient is signed in and has an appointment with the doctor	1) Pick the doctor 2) Click the chat button	A chat window with the doctor opens	Worked as expected the chat window opened
TC-42	Chat with a doctor the patient doesn't have an appointment with (FR-28)	No appointment exists between the patient and the doctor	1) Try to open a chat with the doctor	An access denied message appears	Worked as expected the chat was blocked

TC-43	Send a text message in chat (FR-29)	Chat with the doctor is open	1) Type something 2) Click Send	The message shows up in the chat and gets saved	Worked as expected the message was sent
TC-44	Send an empty message (FR-29)	Chat is open	1) Leave the text box empty 2) Click Send	Nothing gets sent and an alert pops up	Worked as expected the message wasn't sent
TC-45	Send an image or file (FR-30)	Chat is open	1) Click Attach 2) Pick a supported image or file 3) Send	The file shows up in the chat and gets saved	Worked as expected the file was sent
TC-46	Send an unsupported file type (FR-30)	Chat is open	1) Click Attach 2) Pick a file with an unsupported format 3) Send	An "unsupported file type" message appears	Worked as expected the file was rejected
TC-47	Submit a rating (FR-31)	The patient has had a past appointment	1) Open the rating page 2) rate 3) Submit	The rating or complaint is saved and a success message appears	Worked as expected the rating was saved
TC-48	Submit a rating with missing info (FR-31)	The patient has had a past appointment	1) Open the rating page 2) Submit without a rating or any text	A validation message appears and nothing is saved	Worked as expected the error message was shown
TC-49	Doctor gets a message notification (FR-32)	Doctor is signed in and the patient has an appointment with them	1) Have the patient send a message to the doctor 2) Open the doctor's notifications	A notification with the patient's name shows up	Worked as expected the doctor got the notification
TC-50	Doctor views patient messages (FR-33)	There are messages from patients to the doctor	1) Open the messages page 2) Pick a conversation 3) Read through the messages	The patient's messages are shown inside the conversation	Worked as expected the messages were displayed
TC-51	View messages when there aren't any (FR-33)	No conversations exist yet	1) Open the messages page	A "No messages" message is displayed	Worked as expected the empty state appeared
TC-52	Doctor replies to a patient's message (FR-34)	A message exists from a patient who has an appointment	1) Open the conversation 2) Type a reply 3) Send	The reply is saved and shown in the conversation	Worked as expected the reply was sent

3.4.11 Requirements Traceability Matrix:

ID	Title	Dependencies	Coding	Unit Test	Integration Test	System Test	Status	Notes
FR-1	Login	User roles	Auth/AuthenticatedSessionController@store	TC-01, TC-02	TC-01	TC-01	**Done**	Redirect by role
FR-2	Logout	FR-1	Auth/AuthenticatedSessionController@destroy	TC-03	TC-03	TC-03	**Done**	
FR-3	Add Doctor	FR-1	Admin/StaffController@store	TC-04, TC-05	TC-04	TC-04	**Done**	
FR-4	Delete Doctor	FR-3	Admin/StaffController@destroy	TC-06, TC-07	TC-06	TC-06	**Done**	Block if has appointments
FR-5	Add Receptionist	FR-1	Admin/StaffController@store	TC-08	TC-08	TC-08	**Done**	Admin only
FR-6	Delete Receptionist	FR-5	Admin/StaffController@destroy	TC-09	TC-09	TC-09	**Done**	Admin only
FR-7	View Complaints (Admin)	FR-29	Admin/ComplaintController@index	TC-10	TC-10	TC-10	**Done**	
FR-8	Add Specialty to Doctor	FR-3	Admin/StaffController@update	TC-11	TC-11	TC-11	**Done**	
FR-9	Add Patient (Receptionist)	FR-1	Receptionist/PatientController@store	TC-12	TC-12	TC-12	**Done**	No patient login required
FR-10	Search Patient	FR-9	Receptionist/PatientController@index	TC-13	TC-13	TC-13	**Done**	
FR-11	Book Appointment	FR-9, FR-33	Receptionist/Appoint	TC-14, TC-15	TC-14, TC-15	TC-14	**Done**	Prevent double booking

ID	Title	Dependencies	Coding	Unit Test	Integration Test	System Test	Status	Notes
	(Receptionist)		mentController@store					
FR-12	Modify Appointment (Receptionist)	FR-11	Receptionist/AppointmentController@update	TC-16	TC-16	TC-16	**Done**	
FR-13	Cancel Appointment (Receptionist)	FR-11	Receptionist/AppointmentController@cancel	TC-17	TC-17	TC-17	**Done**	
FR-14	Record Payments (Receptionist)	FR-11	Receptionist/PaymentController@store	TC-18	TC-18	TC-18	**Done**	
FR-15	Edit Patient Information (Receptionist)	FR-9	Receptionist/PatientController@update	TC-19	TC-19	TC-19	**Done**	
FR-16	Monthly Reports (Receptionist)	FR-11	Receptionist/ReportController@monthly	TC-20	TC-20	TC-20	**Done**	
FR-17	Send Notification to Doctor (Receptionist)	FR-1	Receptionist/EmergencyNotificationController@store	TC-21	TC-21	TC-21	**Done**	Emergency
FR-18	View Doctor Availability (Receptionist)	FR-33	Receptionist/AppointmentController@availability	TC-22	TC-22	TC-22	**Done**	Shows free slots
FR-19	Patient Self-Registration		Patient/AuthController@register	TC-23, TC-24	TC-23	TC-23	**Done**	Unique email
FR-20	Edit Patient Account	FR-19	Patient/ProfileController@update	TC-25	TC-25	TC-25	**Done**	Owner only

ID	Title	Dependencies	Coding	Unit Test	Integration Test	System Test	Status	Notes
FR-21	Patient Views Doctor Schedules	FR-19, FR-33	Patient/DoctorController@slots	TC-26	TC-26	TC-26	**Done**	
FR-22	Patient Book Appointment	FR-19, FR-33	Patient/AppointmentController@store	TC-27, TC-28	TC-27	TC-27	**Done**	Self-service booking
FR-23	Patient Modify Appointment	FR-22	Patient/AppointmentController@update	TC-29	TC-29	TC-29	**Done**	
FR-24	Patient Cancel Appointment	FR-22	Patient/AppointmentController@cancel	TC-30	TC-30	TC-30	**Done**	
FR-25	Receive Appointment Notifications (Patient)	FR-22	Patient/NotificationController@index	TC-31	TC-31	TC-31	**Done**	Mark as read supported
FR-26	Start Chat with Doctor (Patient)	FR-19, FR-3	Patient/ChatController@startChat	TC-32	TC-32	TC-32	**Done**	
FR-27	Send Text Message (Patient)	FR-26	Patient/ChatController@sendMessage	TC-33	TC-33	TC-33	**Done**	
FR-28	Send Image File (Patient)	FR-26	Patient/ChatController@sendFile	TC-34	TC-34	TC-34	**Done**	Type size validated
FR-29	Submit Rating Complaint (Patient)	FR-22	Patient/RatingController@store	TC-35	TC-35	TC-35	**Done**	Linked to appointment
FR-30	Receive Sent Messages (Doctor)	FR-27	Doctor/ChatController@pollMessages	TC-36	TC-36	TC-36	**Done**	
FR-31	View Messages (Doctor)	FR-30	Doctor/ChatController	TC-37	TC-37	TC-37	**Done**	

ID	Title	Dependencies	Coding	Unit Test	Integration Test	System Test	Status	Notes
			@index, @show					
FR-32	Reply to Messages (Doctor)	FR-31	Doctor/ChatController@reply	TC-38	TC-38	TC-38	**Done**	Text and file reply
FR-33	Set Availability Times (Doctor)	FR-3	Doctor/DoctorController@storeAvailability	TC-39, TC-40	TC-39	TC-39	**Done**	Doctor only
FR-34	View Appointments (Doctor)	FR-11, FR-22	Doctor/DoctorController@appointments	TC-41	TC-41	TC-41	**Done**	Filtered by doctor
FR-35	View Patient Information (Doctor)	FR-34	Doctor/DoctorController@viewPatient	TC-42	TC-42	TC-42	**Done**	Only own patients
FR-36	Add Medical Notes (Doctor)	FR-35	Doctor/DoctorController@storeMedicalNote	TC-43	TC-43	TC-43	**Done**	Doctor only

3.4.12 System architecture:

The proposed healthcare center management system is based on the Model View Controller (MVC) architecture, which is one of the most common and effective software architectures used in developing healthcare and administrative applications. This architecture aims to separate the core components of the system from one another, making development and maintenance easier while increasing the system's flexibility and scalability.

The MVC architecture consists of three main components:

1. Model

- This component is responsible for managing data and the logic associated with it, such as patient data, appointments, doctors, and financial payments.

- It interacts with the database to perform various operations such as retrieving, storing, and updating data.
- The Model is independent of the user interface and control mechanism, allowing code reuse across different parts of the system without modification.

2. View

- This represents the user interface through which end users such as doctors, reception staff, and administrators interact with the healthcare center system.
- It displays data to users in a clear and understandable manner, such as patient lists, appointments, reports, and medical notes.
- The View does not contain any business logic for data processing; instead, it relies on the Controller to update the displayed data.

3. Controller

- It acts as an intermediary between the Model and the View.
- It receives user requests from the interface and processes them by interacting with the Model to retrieve or modify data.
- After processing the request, the Controller updates the View so that the updated information is displayed to the user.

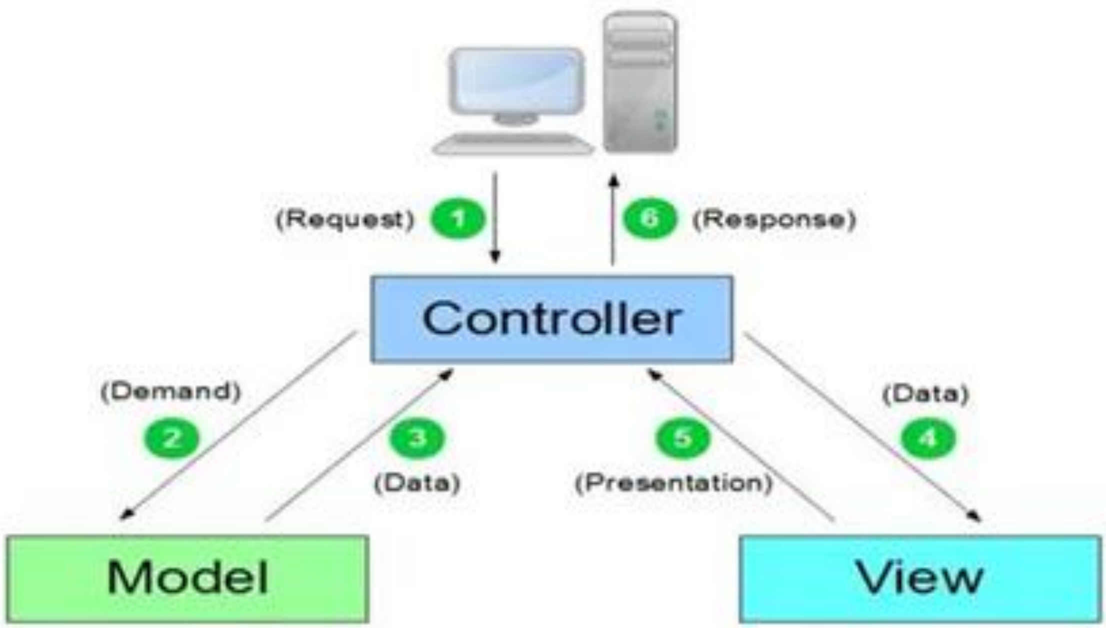
Benefits of Using MVC Architecture in the Healthcare System

1. Ease of Maintenance and Development
Separating the system components allows developers to work on different parts, such as the user interface or patient management, independently without affecting the other system components.

2. Code Reusability
The code written in the Model can be reused in other modules such as appointment management or financial payment management, reducing the time and effort required for development.

3. Performance Improvement
Separating responsibilities among the different components helps improve system performance, as each component can be optimized independently without affecting the others.

4. Ease of Scalability
The MVC architecture allows new features to be added to the system, such as advanced reporting or new record management functionalities, without the need to completely restructure the system.


(Request)
1
6
(Response)
Controller
(Demand)
2
3
(Data)
5
(Presentation)
(Data)
4
Model
View

Chapter Four

Design Study of the Proposed System

4.1 Introduction:

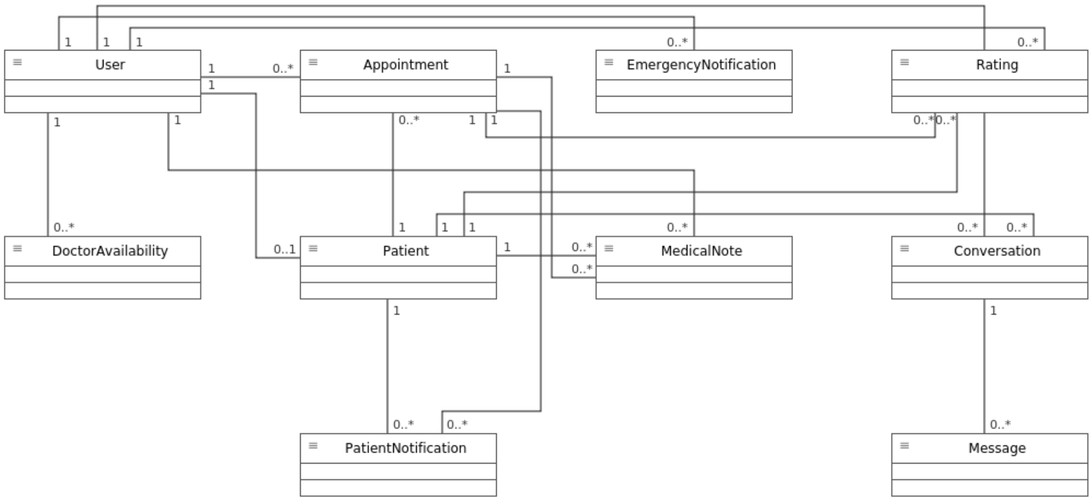
After completing the detailed analysis phase and building the initial diagrams, this chapter will present the design study of the proposed healthcare center management system. The aim of this chapter is to explain the structural and functional models of the system, in addition to clarifying the logical and procedural details that cover all center operations, including patient management, appointments, doctors, and medical notes, along with handling payments and monthly reports.

The chapter also includes design models at different levels, such as user interface designs dedicated to doctors, reception staff, and administrators, in addition to the databases that support all operations to ensure accuracy and fast access to information.

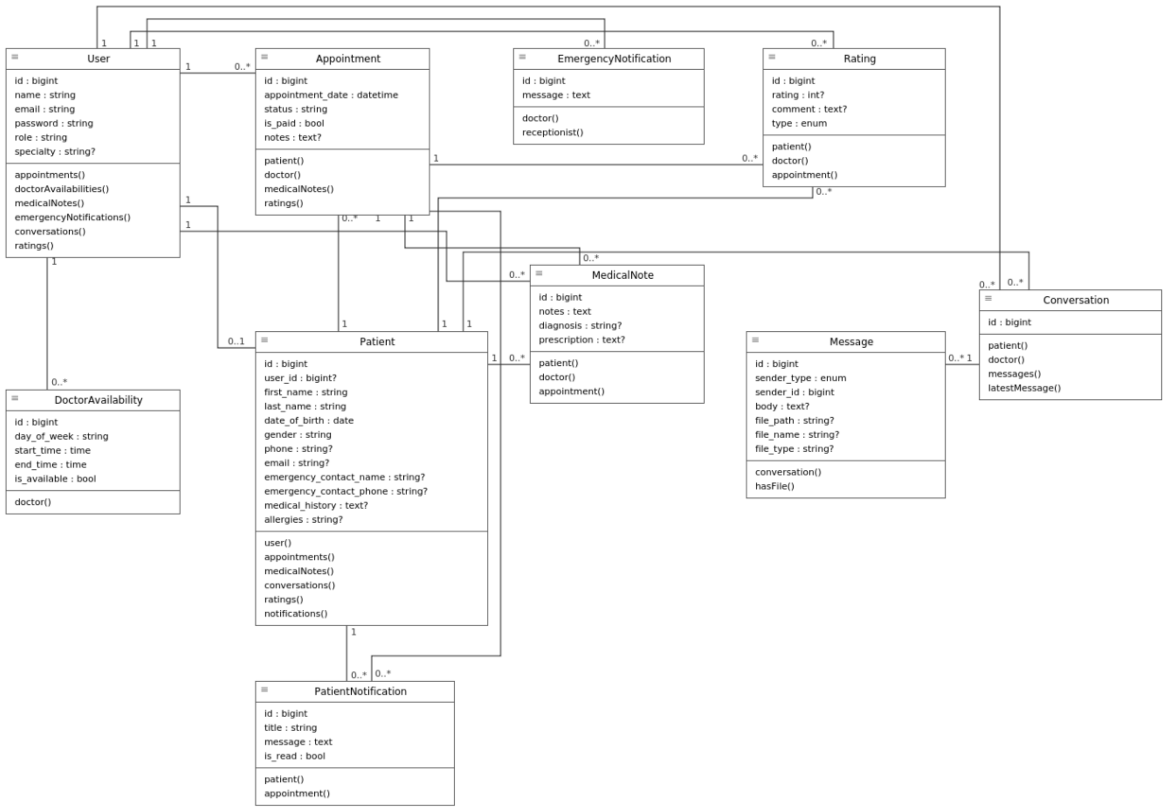
Furthermore, traceability diagrams and Requirement Traceability Matrix (RTM) documents will be included to ensure a clear relationship between requirements and system tests, with the possibility of updating these diagrams in the future to support further development or the addition of new system features.

4.2.1 Class Diagrams:

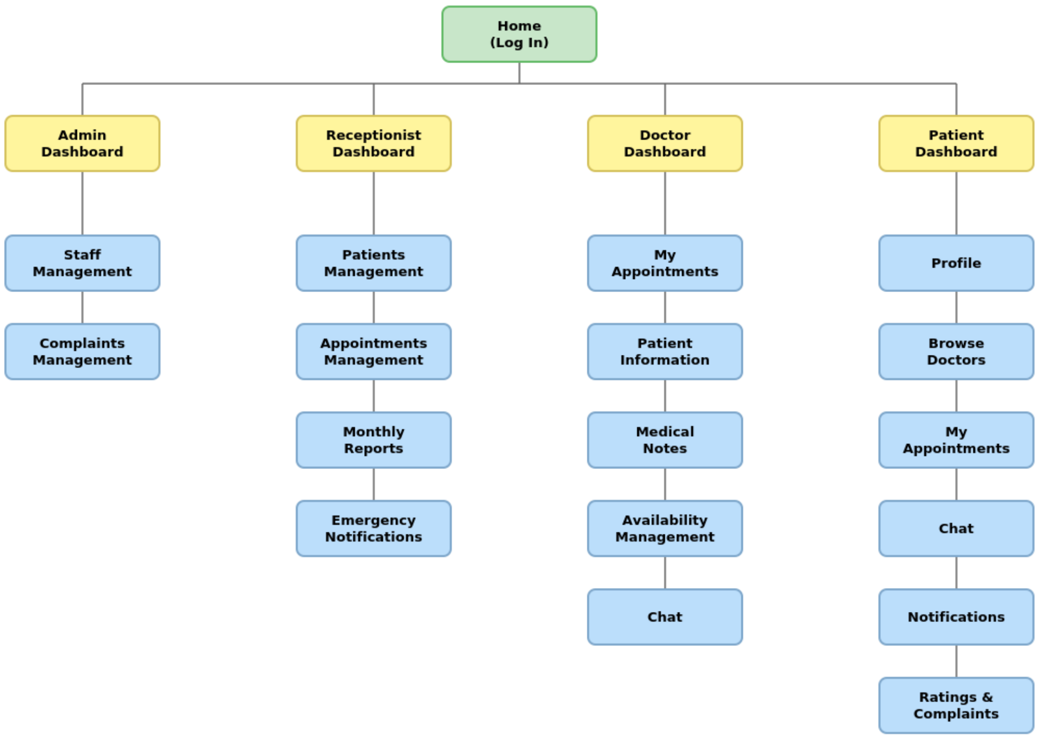
High level :



Low level :

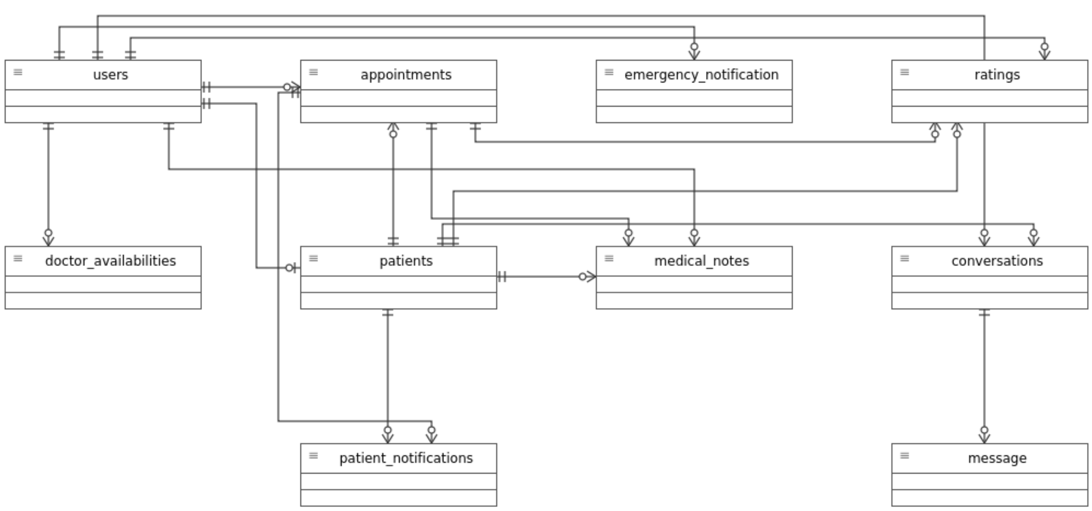

User
id : bigint
name : string
email : string
password : string
role : string
specialty : string?
appointments()
doctorAvailabilities()
medicalNotes()
emergencyNotifications()
conversations()
ratings()
Appointment
id : bigint
appointment_date : datetime
status : string
is_paid : bool
notes : text?
patient()
doctor()
medicalNotes()
ratings()
EmergencyNotification
id : bigint
message : text
doctor()
receptionist()
Rating
id : bigint
rating : int?
comment : text?
type : enum
patient()
doctor()
appointment()
MedicalNote
id : bigint
notes : text
diagnosis : string?
prescription : text?
patient()
doctor()
appointment()
Conversation
id : bigint
patient()
doctor()
messages()
latestMessage()
Message
id : bigint
sender_type : enum
sender_id : bigint
body : text?
file_path : string?
file_name : string?
file_type : string?
conversation()
hasFile()
Patient
id : bigint
user_id : bigint?
first_name : string
last_name : string
date_of_birth : date
gender : string
phone : string?
email : string?
emergency_contact_name : string?
emergency_contact_phone : string?
medical_history : text?
allergies : string?
user()
appointments()
medicalNotes()
conversations()
ratings()
notifications()
DoctorAvailability
id : bigint
day_of_week : string
start_time : time
end_time : time
is_available : bool
doctor()
PatientNotification
id : bigint
title : string
message : text
is_read : bool
patient()
appointment()

4.2.2 System interface tree:

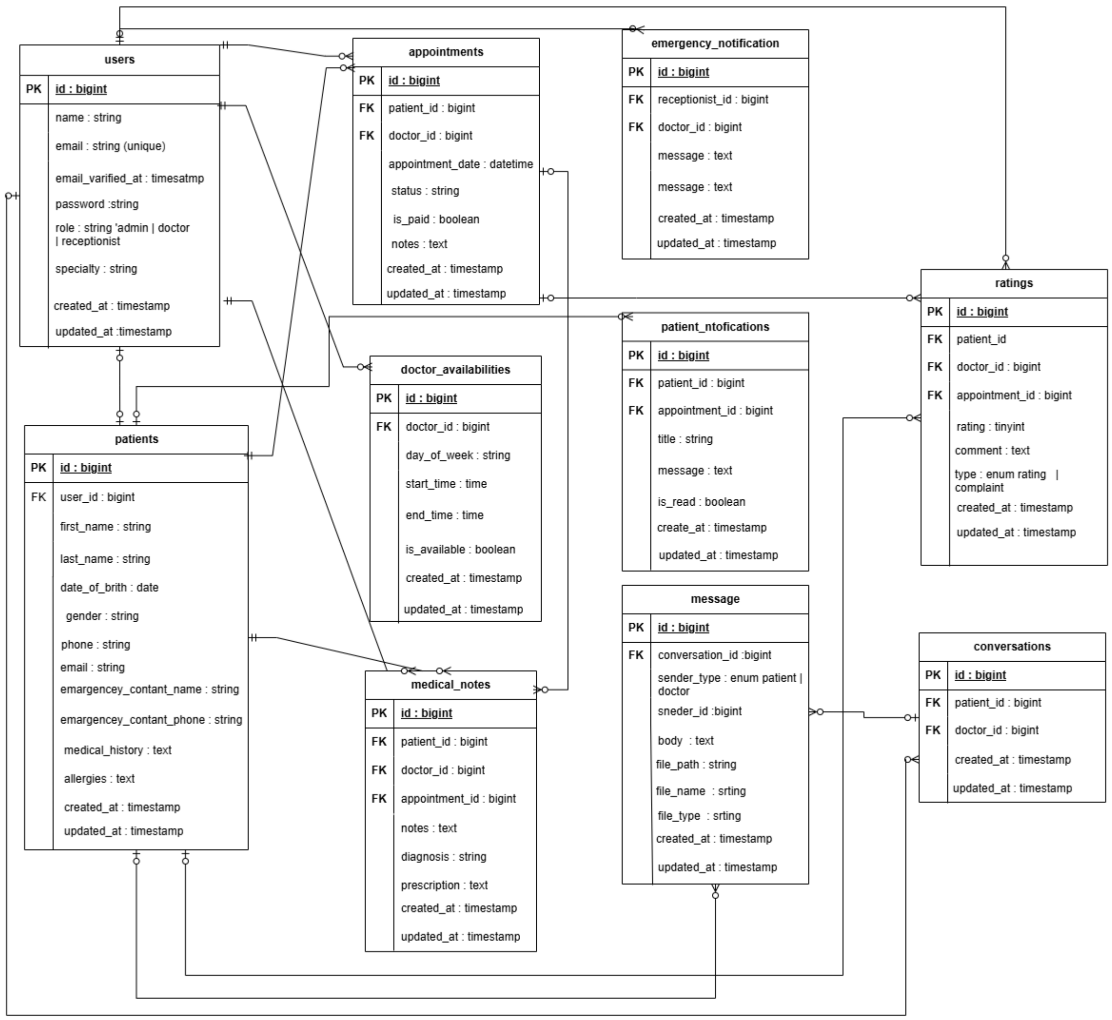


4.2.3 ERD Diagrams:

High level:



Low level:


users
PK id : bigint
name : string
email : string (unique)
email_varified_at : timesatmp
password :string
role : string 'admin | doctor | receptionist
specialty : string
created_at : timestamp
updated_at :timestamp
appointments
PK id : bigint
FK patient_id : bigint
FK doctor_id : bigint
appointment_date : datetime
status : string
is_paid : boolean
notes : text
created_at : timestamp
updated_at : timestamp
emergency_notification
PK id : bigint
FK receptionist_id : bigint
FK doctor_id : bigint
message : text
message : text
created_at : timestamp
updated_at : timestamp
ratings
PK id : bigint
FK patient_id
FK doctor_id : bigint
FK appointment_id : bigint
rating : tinyint
comment : text
type : enum rating | complaint
created_at : timestamp
updated_at : timestamp
patient_ntofications
PK id : bigint
FK patient_id : bigint
FK appointment_id : bigint
title : string
message : text
is_read : boolean
create_at : timestamp
updated_at : timestamp
doctor_availabilities
PK id : bigint
FK doctor_id : bigint
day_of_week : string
start_time : time
end_time : time
is_available : boolean
created_at : timestamp
updated_at : timestamp
patients
PK id : bigint
FK user_id : bigint
first_name : string
last_name : string
date_of_brith : date
gender : string
phone : string
email : string
emargencey_contant_name : string
emargencey_contant_phone : string
medical_history : text
allergies : text
created_at : timestamp
updated_at : timestamp
medical_notes
PK id : bigint
FK patient_id : bigint
FK doctor_id : bigint
FK appointment_id : bigint
notes : text
diagnosis : string
prescription : text
created_at : timestamp
updated_at : timestamp
message
PK id : bigint
FK conversation_id :bigint
sender_type : enum patient | doctor
sneder_id :bigint
body : text
file_path : string
file_name : srting
file_type : srting
created_at : timestamp
updated_at : timestamp
conversations
PK id : bigint
FK patient_id : bigint
FK doctor_id : bigint
created_at : timestamp
updated_at : timestamp

4.3 Summary:

Chapter Four included a presentation of the overall design of the healthcare center management system, where a set of design diagrams was provided to illustrate the structural and functional architecture of the system, contributing to a better understanding of its operation mechanism and core components.

The chapter also reviewed the Requirement Traceability Matrix (RTM), which demonstrates the relationship between the various system requirements and the design and testing phases, ensuring that all functional and non-functional requirements identified during the analysis phase are fulfilled.

This chapter helps provide a solid foundation for moving to the implementation phase while ensuring the system’s maintainability and scalability in the future through a clear and organized design.

Chapter Five
Practical Application

5.1 Introduction:

After completing the creation of diagrams, relationships, and clarifying the algorithms used in the healthcare center management system, this chapter will present the design of the system interfaces and the tools used in its development.

This chapter includes an explanation of the software environment, tools, and technologies used to build the system, along with details about the design of user interfaces and the database, providing a comprehensive understanding of the system's software and procedural architecture.

5.2 Tools and techniques used

5.2.1 System Tools

1. Draw.io:
It is an open-source software tool used for creating and editing UML models, which is a standard modeling language for representing and designing software systems in a visual way. Draw.io supports a wide range of diagrams, including:

- **Class Diagrams:** to represent the system structure in terms of classes, interfaces, and their relationships.
- **Sequence Diagrams:** to represent interactions between objects in a time sequence.
- **Activity Diagrams:** to represent the flow of processes and activities within the system.
- **Use Case Diagrams:** to represent interactions between actors and the system.

2. XAMPP:
A free and open-source software package designed to simplify web application development. It includes:

- **X (Cross-platform):** ability to run on multiple operating systems such as Windows, Linux, and Mac.
- **Apache:** the web server used to host applications.
- **MySQL / MariaDB:** database management system.
- **PHP and Perl:** programming languages used for web application development.

3. Visual Studio Code (VSCode):
An open-source code editor developed by Microsoft, used for writing and editing source code. It is highly flexible and supports many programming languages and libraries, making it a powerful tool for application development.

5.2.2 Used technologies:

1. HTML (Hypertext Markup Language):
A markup language used to create web pages, providing the basic structure of content displayed on the internet.

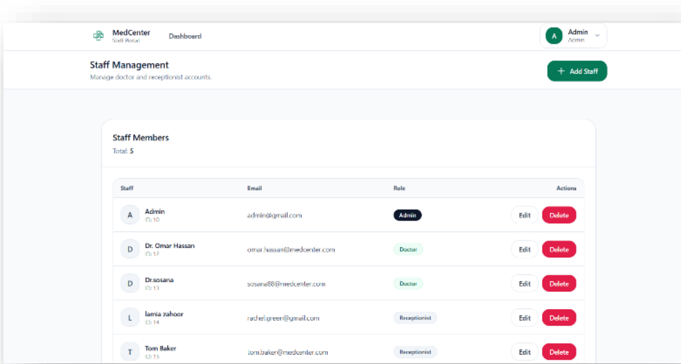
2. CSS (Cascading Style Sheets):
A styling language used to format HTML pages, such as controlling colors, fonts, spacing, and the overall layout of the page.

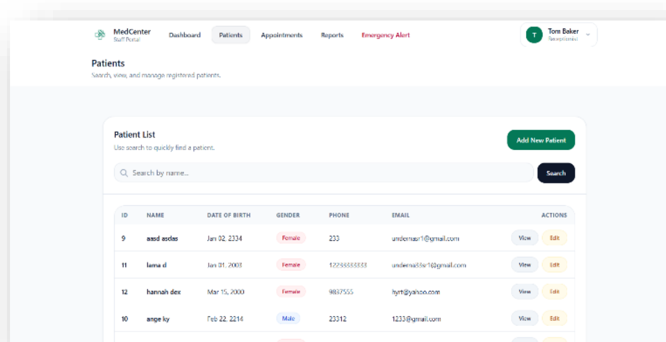
3. JavaScript (JS):
A dynamic programming language used to add interactivity to web pages and enhance the user experience.

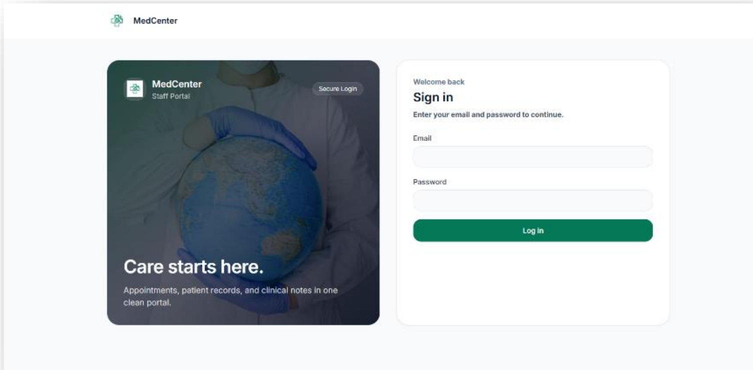
4. PHP (Hypertext Preprocessor):
An open-source scripting language used to create dynamic and interactive web pages. It is widely used in web application development.

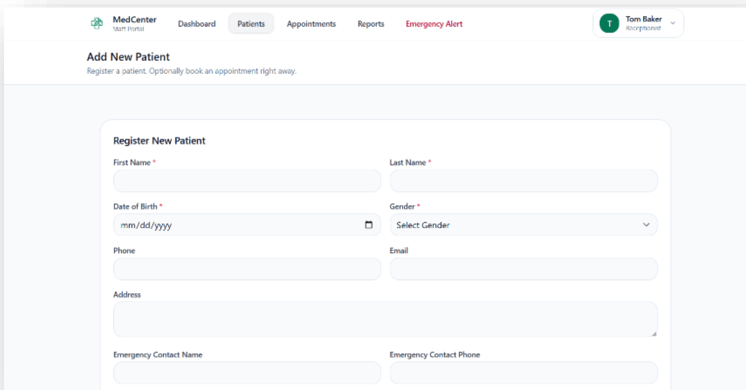
5. Laravel:
An open-source PHP framework used for web application development. It provides tools and features that help developers write clean and well-structured code. Laravel is considered one of the most popular frameworks in the PHP community.

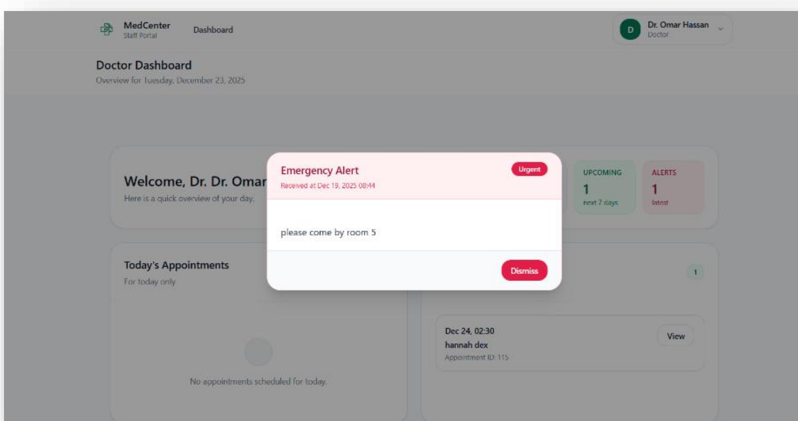
5.3 System Interfaces:






MedCenter
MedCenter
Staff Portal
Secure Login
Care starts here.
Appointments, patient records, and clinical notes in one clean portal.
Welcome back
Sign in
Enter your email and password to continue.
Email
Password
Log in


MedCenter
Staff Portal
Dashboard
Patients
Appointments
Reports
Emergency Alert
Tom Baker
Add New Patient
Register a patient. Optionally book an appointment right away.
Register New Patient
First Name *
Last Name *
Date of Birth *
mm/dd/yyyy
Gender *
Select Gender
Phone
Email
Address
Emergency Contact Name
Emergency Contact Phone


MedCenter
Staff Portal
Dashboard
Dr. Omar Hassan
Doctor
Doctor Dashboard
Overview for Tuesday, December 23, 2025
Welcome, Dr. Dr. Omar
Here is a quick overview of your day.
Emergency Alert
Urgent
please come by room 5
Dismiss
UPCOMING
1
next 7 days
ALERTS
1
Today's Appointments
For today only
No appointments scheduled for today.
Dec 24, 02:30
hannah dex
View

5.4 Summary:

This chapter focuses on explaining all the tools and technologies used in building the healthcare center management system, with a clarification of the role of each tool in developing the different components of the system.

It also presents the interfaces designed and developed for the system, including those for doctors, reception staff, and administrators, to facilitate daily operations and manage data effectively.

This chapter enables developers and users to understand the software environment and technical architecture of the system, and it forms a foundation for the implementation phase, with the possibility of making future improvements or additions to the tools or user interfaces.

Chapter Six
Project Management

Project Description:

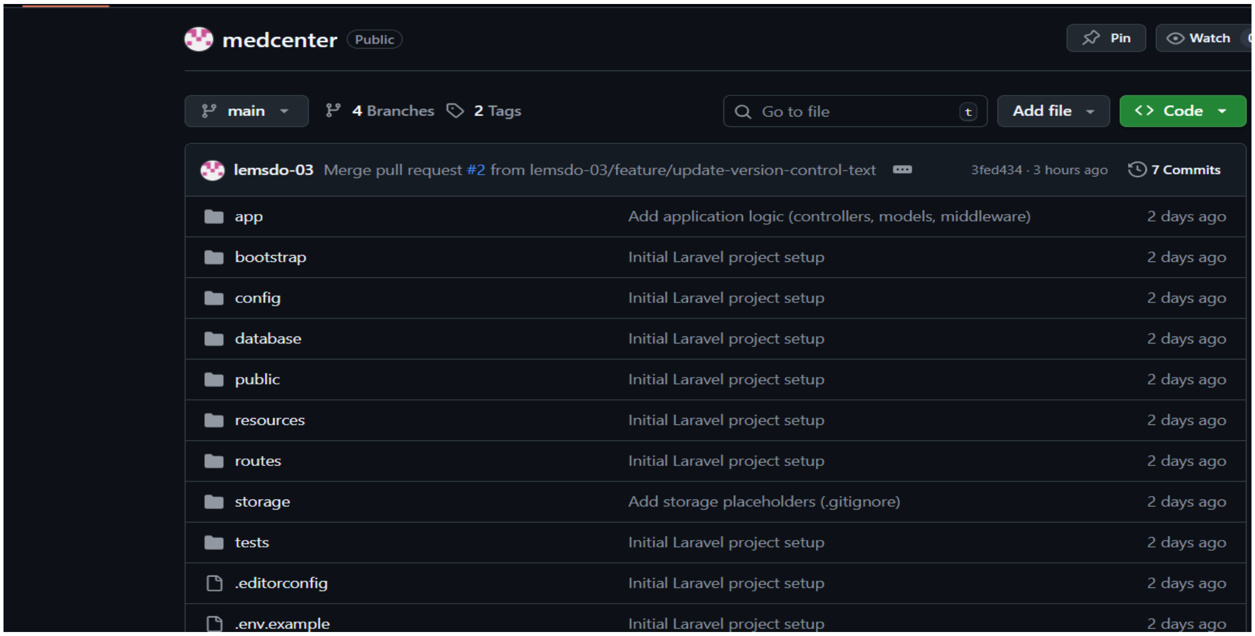
MedCenter is a web application built with the Laravel framework The project aims to organize the daily operations of a clinic by managing patients, appointments, staff accounts, and appointment payment status. The system uses role-based access control to define what each user (Admin, Doctor, Receptionist or Patient) is allowed to do

Git:

Version control for the project is managed using Git

GitHub file hosting

1. Link: https://github.com/lemsdo-03/medcenter
2. Purpose: Contains the full MedCenter project (backend + frontend).
3. Structure:
    - app/ (controllers, models, middleware)
    - routes/ (web routes)
    - database/ (data migrations and seeds)
    - resources/ (Blade views + assets)
    - Documentation files such as VERSION_CONTROL.md



3 .Branch and Merge Strategy

Branches Used

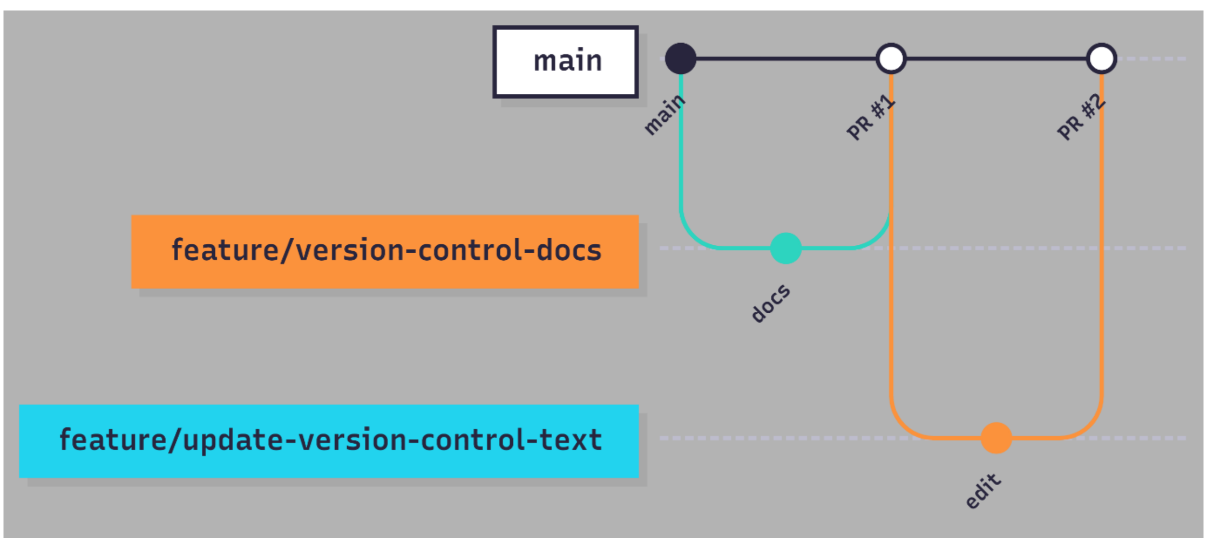
- main: the stable branch
- feature: temporary branches for features

Branch Workflow

1. A new feature branch is created from the main branch.
2. Changes are made and saved in the feature branch.
3. A pull request is created to merge the feature branch into the main branch.
4. After merging, the main branch contains the updated and stable version.

Examples of branches from this project

- feature/version-control-docs (merged into main)
- feature/update-version-control-text (merged into main).

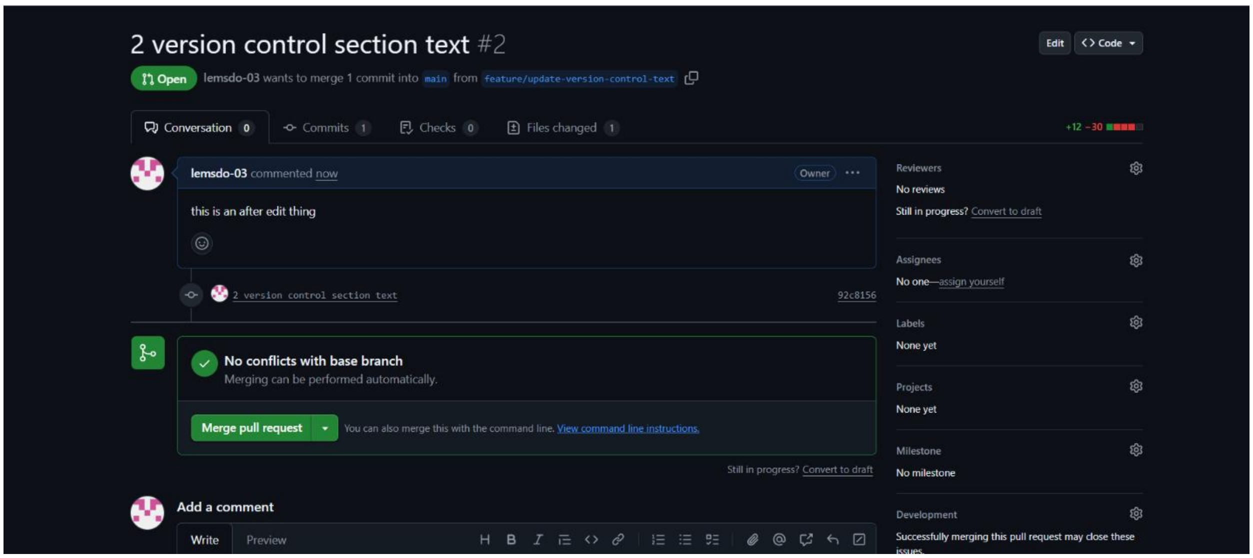


4. Team Members and Responsibilities

The project was developed by a team consisting of two people. All team members participated in the stages of system analysis and architectural design, including requirements analysis, system modeling, and defining the overall system architecture.

Pull Request Guide:

Pull requests were used to merge feature branches into the main branch to keep changes organized and traceable..



5. Development Workflow

The development workflow of the MedCenter project follows a simplified task-based approach to keep changes organized and traceable using pull requests on GitHub.

The workflow can be summarized as follows:

1. Identify a development task: A new task or feature to be developed is identified.
2. Create a branch: A new branch named feature/* is created from the main branch.
3. Implementation and local testing: Changes are implemented and tested locally on the new branch.

4. Save changes: Commits are created with clear messages describing the updates made.
5. Push the branch: The feature branch is pushed to the GitHub repository.
6. Create a pull request: A pull request is created to merge the feature branch into the main branch.
7. Review and merge: After reviewing the changes and obtaining approval, the pull request is merged into the main branch.

6 .Tags and Version Management

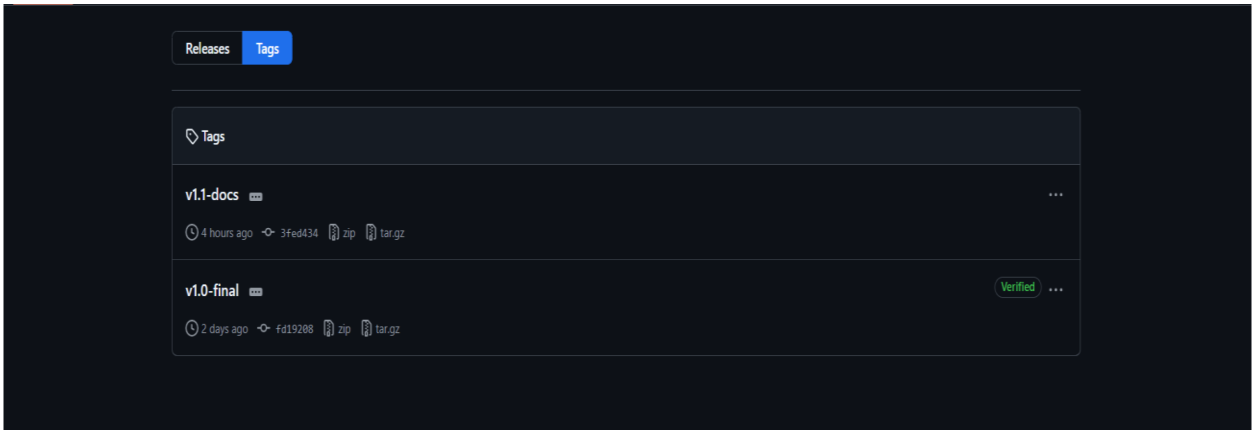
Git tags were used to identify important milestones of the project and stable delivery versions in the MedCenter repository.

Tagging helps to:

- Identify release points.
- Improve traceability.
- Enable the team to revert to a specific stable state of the code when needed.

The tags created:

- v1.0-final: The final baseline delivery version for the MedCenter project.
- v1.1-docs: An updated version for version control documentation and report enhancements.



Chapter Seven

Conclusions and References

7.1 Conclusion

This project presented the design and implementation of the Medical Center Management System, a web-based platform built with Laravel and MySQL that organizes the daily administrative and medical operations of a medical center. The system covers four user roles Administrator, Doctor, Receptionist, and Patient each with its own dashboard, permissions, and workflows.

Through this system, medical centers can replace manual processes with a single integrated platform. The system enables receptionists to register patients and book appointments without scheduling conflicts, allows doctors to manage their availability and write medical notes, gives patients self-service access to appointments and chat with their doctors, and provides administrators with staff management and complaint-monitoring tools.

The development followed an iterative and incremental methodology, with the codebase organized using the Model-View-Controller (MVC).

Functional and non-functional requirements were systematically defined, traced through a Requirements Traceability Matrix, and verified through a structured set of test cases.

7.2 Future Work

While the current system delivers the core functionality identified in the requirements specification, several areas remain open for future development:

- Real-time chat using WebSockets instead of HTTP polling, to eliminate the delay between sent and received messages.

- A full billing and payment processing module, beyond the current paid/unpaid flag on appointments.
- Email and SMS notifications for appointment reminders.
- A mobile-friendly progressive web app (PWA) version for patients.
- Analytical dashboards with charts and trends, beyond the current monthly reports.
- Multi-language support, including a fully Arabic interface.

7.2 Future Work

While the current system delivers the core functionality identified in the requirements specification, several areas remain open for future development:

A mobile version of the application, so patients and staff can access their dashboards on the go.

- Multi-language support, including a fully Arabic interface in addition to the current English one.
- Integration of an AI chatbot into the system to assist patients with common questions, appointment guidance, and basic medical information, reducing the workload on reception staff.

7.3 Final Remarks

The development of this project provided practical experience in full-stack web development, requirements engineering, database design, and team-based version control using Git and GitHub. Beyond its technical outcomes, the project demonstrates how software can directly improve healthcare administration by reducing manual effort, preventing errors, and giving each stakeholder a clearer view of the information they need.

7.4References

Practo | Video Consultation with Doctors, Book Doctor Appointments, Order Medicine, Diagnostic Tests

https://heltog.com

https://www.angelinfotech.org